分类号 TP391　　单位代码 11395

密　级 公开　　学　　号 2108210130

榆林学院

学生毕业设计（论文）

题　　目	Python 语言类图书销售和评论数据分析
作　　者	李卓妍
院　(系)	信息工程学院
专　　业	计算机科学与技术
指导教师	
答辩日期	年　月　日

榆　林　学　院

毕业设计诚信责任书

本人郑重声明：所呈交的毕业设计，是本人在导师的指导下独立进行研究所取得的成果。毕业设计中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。尽我所知，除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经公开发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

本人毕业设计与资料若有不实，愿意承担一切相关的法律责任。

论文作者签名:

年　　月　　日

摘要

近些年来，随着人工智能、大数据分析和科学计算等领域的快速发展，编程语言已经成为了计算机领域技术实现的重要的工具，Python 语言凭借着其简洁的语法结构、丰富的第三方库生态以及跨平台特性，已成为全球最受欢迎的编程语言之一，随着 Python 编程语言在各行各界的广泛应用，针对 Python 语言类技术图书的市场表现以及读者的反馈研究具有着一定的商业价值和学术意义。

本研究选取国内领先的图书电商平台当当网作为数据采集源，利用 Python 编写网络爬虫程序，通过 requests 库发送 HTTP 请求获取网页数据，结合 BeautifulSoup 库解析 HTML 页面，对书名、作者、出版社、评论数量、星级评分、出版年份、定价、销量等关键特征值进行系统性采集。为保证数据质量，采用 Pandas 库对原始数据进行清洗，处理缺失值、重复值以及异常值，并通过正则表达式对文本数据进行标准化处理，如统一书名格式、规范出版社名称。运用 Matplotlib 与 Seaborn 数据可视化库，实现多维数据的可视化分析。

在出版社数据分析环节，分析了读者满意度、定价策略、图书销量以及出版量，识别出哪些出版社的质量较高、哪些出版社的销量较高、哪些出版社的出版图书量较多并研究了各出版社的平均定价，并对星级、价格、评论数量这三个特征值进行了相关性分析，挖掘特征值之间的内部联系。

在图书销量数据分析部分，对作者的作品数量进行统计筛选出高产作者，对作者的作品评论总数进行平均筛选出高热度作者，对作者作品获得的平均星级进行统计筛选出高质作者。将图书按照编程基础、数据分析、机器学习等等应用领域进行分类标注。并采用相关性分析完成了图书销量与各特征值之间的关系分析。基于处理后的数据，应用梯度提升机器学习模型，构建销量预测模型，并利用均方误差（MSE）等指标对模型性能进行评估，实现对 Python 语言类图书销量的精准预测。

针对用户情感分析部分，首先对用户评论数据进行分词、去除停用词等预处理，使用 K-means 聚类算法对图书品类进行聚类分析，通过肘部法则确定最优聚类数，精准识别出热销图书品类。在此基础上，构建多维度评论分析框架：运用 TF-IDF 算法对评论内容进行特征提取，将文本转化为数值型特征向量，结合 SVM 分类器对评论进行情感分类，划分出积极评论和消极评论的情感类别。同时，利用 LDA 主题模型对评论进行主题建模，发掘出文本中蕴含的情感倾向及具体情感内容。

关键词：Python 类图书；销售数据；价格预测；情感分析；数据分析

ABSTRACT

In recent years, with the rapid development of fields such as artificial intelligence, big data analysis, and scientific computing, programming languages have become important tools for technical implementation in the computer field. Python, with its concise syntax, rich third-party library ecosystem, and cross-platform features, has become one of the most popular programming languages worldwide. With the wide application of Python programming language in various industries, the market performance of Python-related technical books and reader feedback research have significant commercial value and academic significance.

This study selected Dangdang, a leading domestic online book platform, as the data source. A web crawler program was written in Python to send HTTP requests using the requests library to obtain web page data. The BeautifulSoup library was used to parse HTML pages, and key feature values such as book titles, authors, publishers, review counts, star ratings, publication years, prices, and sales volumes were systematically collected. To ensure data quality, the Pandas library was used to clean the raw data, handling missing values, duplicates, and outliers, and standardizing text data through regular expressions, such as unifying book titles and standardizing publisher names. The Matplotlib and Seaborn data visualization libraries were used to conduct multi-dimensional data visualization analysis.

In the analysis of publisher data, reader satisfaction, pricing strategies, book sales, and publication volumes were analyzed to identify publishers with higher quality, higher sales, and larger publication volumes, and to study the average pricing of each publisher. Correlation analysis was conducted on star ratings, prices, and review counts to explore the internal connections among these feature values.

In the analysis of book sales data, the number of works by authors was counted to identify high-yield authors, the average number of reviews for each author's works was calculated to identify high-heat authors, and the average star ratings of authors' works were counted to identify high-quality authors. Books were classified and labeled according to application fields such as programming basics, data analysis, and machine learning. Correlation analysis was used to analyze the relationship between book sales and various feature values. Based on the processed data, a gradient boosting machine learning model was applied to build a sales prediction model, and the model performance

was evaluated using metrics such as mean squared error (MSE) to achieve accurate sales predictions for Python-related books.

For user sentiment analysis, user review data was preprocessed by tokenization and removal of stop words. The K-means clustering algorithm was used to cluster book categories, and the elbow method was applied to determine the optimal number of clusters to accurately identify popular book categories. On this basis, a multi-dimensional review analysis framework was constructed: the TF-IDF algorithm was used to extract features from review content, converting text into numerical feature vectors. The SVM classifier was used to classify reviews into positive and negative sentiments. Additionally, the LDA topic model was employed to model the themes of reviews, uncovering the emotional tendencies and specific emotional content contained in the text.

Keywords: Python-related books, sales data, price prediction, sentiment analysis; data analysis

目录

摘要 I
ABSTRACT II
第 1 章 绪论 1
1.1 研究背景和研究意义 1
1.1.1 国内背景 1
1.1.2 国际背景 2
1.1.3 研究意义 2
1.2 研究思路与内容 3
1.2.1 研究方法 3
1.2.2 研究内容 3
第 2 章 相关理论及技术介绍 5
2.1 Python 语言 5
2.2 网络爬虫技术 5
2.3 分析算法介绍 6
第 3 章 数据采集 8
3.1 数据源 8
3.2 图书基础数据采集 8
3.3 用户评论数据爬取 15
3.4 数据预处理 20
第 4 章 数据分析与可视化实现 25
4.1 出版社数据分析与可视化 25
4.1.1 出版社读者满意度分析 25
4.1.2 出版社定价分析 26
4.1.3 出版社图书销量分析 27
4.1.4 出版社图书出版量分析 29
4.1.5 相关性分析 30
4.2 图书销量数据分析 32
4.2.1 作者分析 32
4.2.2 图书的应用领域分析 37
4.2.3 图书销量的相关性分析 42
4.2.4 图书销量的预测分析 47

4.3 热销图书用户评论数据情感分析 51
4.3.1 基于 K-means 聚类的热销图书分析 51
4.3.2 基于 LDA 的评论主题分布分析 54
4.3.3 热销书用户评论情感分析 58
第 5 章 总结 63
参考文献 64
致谢 66

第 1 章 绪论

1.1 研究背景和研究意义

1.1.1 国内背景

现如今随着数字化时代的快速发展，图书销售与评论数据的分析已经成为各个出版行业和电商平台优化运营、提升用户体验的核心研究方向。Python 语言凭借其强大的数据处理能力和丰富的开源工具库，在图书销售与评论分析领域展现了显著的技术优势。国内学者基于 Python 构建图书销售管理系统，强调大数据技术在库存管理、用户画像构建和精准推荐中的应用。

图书销售行业正面临数字化转型的迫切需求。传统的销售模式容易存在信息缺失、管理低效、用户行为分析不足等问题。然而电商平台的兴起（如淘宝、京东、豆瓣等等网购平台）积累了海量销售数据（如销量、评分、评论），这些数据蕴含着市场趋势、用户偏好和产品改进方向的关键信息。例如，淘宝图书销售数据可以揭示消费差异和热门品类的分布，帮助出版社制定精准的营销策略，又比如，豆瓣评论数据则能挖掘读者情感倾向，为图书内容方面的优化提供依据。

此外，消费者对于个性化推荐服务的需求日益增长，基于用户行为的推荐算法可显著提升购买概率，而实现这一目标依赖于对销售与评论数据的深度分析。

尽管已有部分研究关注图书销售数据分析，但仍存在以下局限性：

1.数据不足：仅聚焦单一数据（如价格或销量），而缺乏了对于评论的深度分析。

2.动态性与实时性欠缺：传统分析多基于静态历史数据，难以适应快速变化的用户和市场需求。

3.用户画像深度不足：现有推荐算法对用户行为的动态追踪和上下文关联分析较少，导致推荐结果精准度有限。

随着数字化进程的加速推进，我国的图书销售行业正经历从传统模式向数据驱动的智能化模式进行转型。在国家《“十四五”数字经济发展规划》的指导下，数据要素的市场化配置被提升至战略高度，并且鼓励企业利用大数据技术优化运营效率。在此背景下，国内的电商平台积累了海量的图书销售数据与评论数据。这些数据不仅包含销量、价格、评分等有效信息，还涵盖用户评论这种非结构化的文本信息，为分析市场趋势和用户行为提供了丰富的数据来源。本研究既源于行业数字化转型的迫切需求，也得益于 Python 在数据处理领域的独特优势。该研究可为电商行业提供科学的分析与支持，据此推动智能化、个性化的图书服务的建设。

1.1.2 国际背景

在全球范围内，图书的销售与评论数据分析已经成为出版业行业和电商平台的核心竞争力之一。国际电商巨头（如亚马逊、Barnes & Noble）和社交阅读平台通过用户行为追踪和分析，构建了成熟的个性化推荐系统。例如，亚马逊基于用户浏览、购买历史以及评论情感等等数据，利用协同过滤算法实现“猜你喜欢”功能，很好地发展了个性化服务这一功能；又例如，Goodreads 则通过 TF-IDF 和深度学习模型对数据评论数据进行分析，从而生成阅读兴趣图谱，推动对用户的精准营销。国外学者较早将 Python 与大数据技术结合，协同过滤算法和混合推荐系统的研究在亚马逊等平台的应用中较为成熟，为国内推荐系统设计提供了参考。同时，国外研究更倾向于融合社会学、经济学与数据科学。

然而，国际研究也存在一些缺点：

1. 合规风险：欧盟《通用数据保护条例》对用户数据采集进行了严格的限制，导致部分平台数据获取难度增加。

2. 文化差异：跨国分析需处理语言、评分标准（如五星制 vs 十分制）等等方面的地域性差异，传统模型的能力不足以解决这个问题。

目前国际前沿研究正探索联邦学习与差分隐私技术，在合法合规的前提下实现跨平台数据协作。例如，Springer Nature 联合多家出版社利用 Python 联邦学习框架，在不共享原始数据的情况下联合训练销量预测模型，这样能做到既保护隐私的同时又提升预测精度。

1.1.3 研究意义

Python 作为当前最热门的编程语言之一，其相关图书的销售数据和用户评论能够反映技术社区的关注焦点、知识需求演变及新兴领域（如人工智能、数据分析）的发展趋势。通过数据挖掘，可揭示技术推广与知识传播的规律，为计算机科学、教育技术学等领域提供实证研究基础。

而评论数据包含读者对内容质量、写作风格、实用性等方面的主观反馈，结合情感分析、主题建模等自然语言处理技术，可构建用户评价与图书销量之间的关联模型，丰富信息科学和社会计算领域的研究方法。

分析图书的应用领域（如基础语法、机器学习、Web 开发等），可以识别教育市场中不同受众（如学生、工程师、科研人员）的差异化需求，为教育机构定制课程体系、编写教材提供数据支持。出版社可据此调整选题方向，避免盲目跟风。

通过销售数据分析，出版社可了解到市场趋势，能推动图书快速迭代，提升市场竞争力。结合价格区间和用户评论等等信息，可制定差异化营销策略，使得定价

策略科学化，为图书的定价提供数据支持。也可以用于增强读者的体验，构建“数据-需求”的闭环设计。

通过图书的评论数据进行用户的情感分析，可以用来进行个性化推荐服务的升级优化。

本研究可以为出版业出版以及电商平台的销售提供一定的参考价值，为出版、教育、销售、技术等方面提供分析思路，也在方法论层面推动多学科融合，同时引发对数据伦理、技术传播效率等深层问题的探讨。通过分析 Python 图书市场，研究者能够以小见大，揭示知识流动规律与用户行为特征。

1.2 研究思路与内容

首先进行数据采集和数据预处理，通过网络爬虫技术获取当当网 Python 类图书的销售数据（价格、销量、出版社、星级等）和用户评论数据（评分、文本、评论时间等）。然后对数据进行清洗，删除价格缺失记录，过滤异常值。

随后进行数据分析过程，对出版社的读者满意度、定价、图书销量进行分析并研究了三者的相关性，在图书销量分析部分，对高产作者、图书的应用领域、图书销量进行分析并利用梯度提升模型预测图书的销量，使用了 K-means 聚类算法聚类出热销图书，并基于此使用 TF-IDF 方法、SVM 算法和 LDA 算法对用户评论进行情感分析。

1.2.1 研究方法

本研究主要采用了以下研究方法：

文献研究法：通过查阅国内外 Python 图书平台、可视化分析等相关文献，了解相关理论知识和发展趋势，为本课题的研究提供方法和思路。

网络数据挖掘法：从相关网站收集 Python 文字类图书的相关数据，例如作者、出版社、价格、评论等等，为研究提供数据支撑。

可视化分析法：根据所获取的数据制作可视化图表，例如不同出版社的图书平均星数统计图、不同出版社价格排名统计图，不同出版社的评论总数柱状统计图、图书特征值相关性矩阵图等等，根据图表能够更加清晰直观地了解和对比数据。

1.2.2 研究内容

本研究基于面向对象的设计理念，对当当网 Python 语言类图书的外部特征数据以及用户评论信息进行了采集与分析。通过分页式爬取技术和数据遍历算法实现了高效的数据获取，并基于所得数据进行分析与可视化实现。

研究重点考察了出版社读者满意度、图书定价与销量之间的关联性，其中在销量分析模块，针对高产出作者、图书应用领域及销售表现进行了深入探讨，并应用梯度提升模型实现销量预测。同时采用 K-means 聚类算法识别热销图书类别，结合 TF-IDF 特征提取方法、SVM 分类器和 LDA 主题模型对用户评论进行情感分析。

第 2 章 相关理论及技术介绍

2.1 Python 语言

Python 语言是一种高级的、通用的编程语言，它拥有着非常简洁易读的语法结构，相对来说比较简单易学，代码可读性很高。同时也兼顾着强大的功能，它拥有着丰富的第三方库生态以及跨平台特性，内置大量的实用模块，并且支持 Windows、Linux 等等操作系统，涵盖了 Web 开发、数据分析等诸多领域，目前来看已经成为全球最受欢迎的编程语言之一，Python 编程语言已经在各行各界广泛应用。

2.2 网络爬虫技术

Python 网络爬虫技术是一种基于 Python 编程语言的自动化数据采集程序，通过模拟浏览器行为访问互联网资源并提取目标信息。其工作原理可分为以下四个关键步骤：

1. 请求发送：爬虫向目标服务器发送 HTTP 请求，模拟用户访问行为。

2. 响应解析：获取 HTML/JSON 响应后，使用 BeautifulSoup、lxml 等库解析文档结构，通过 XPath 或 CSS 选择器定位目标数据。高级场景需处理反爬机制如验证码、IP 封锁等。

3. 数据提取：从解析后的文档中抽取结构化信息，常用正则表达式或专用解析器处理复杂格式。

4. 存储处理：将清洗后的数据存入数据库（MySQL/MongoDB）或文件系统（CSV/JSON）中，以供后续的分析和使用。

典型 Python 爬虫系统包含以下模块化设计：

1. 调度中心：URL 管理器维护待抓取队列，采用优先级算法控制爬取顺序、任务分发模块实现负载均衡，支持断点续爬。

2. 下载器集群：代理 IP 池实现 IP 轮换，User-Agent 池模拟多设备访问、自适应限速机制避免触发反爬策略。

3. 存储系统：分布式文件存储（HDFS/S3）、结构化数据库（PostgreSQL/Elasticsearch）。

处理过程包括请求管理、编码处理、反爬措施、分页控制、数据合并和持久化存储等关键技术，各个技术模块协同作用。

请求管理可以有效地保障数据采集的连续性和稳定性，减少因请求异常而导致的数据缺失。编码处理模块具有很好的准确率与兼容性，可以解决中文网页乱码问题，保持原始数据的准确性和完整性。反爬措施可以实现多重防护、动态调整以及低成本维护，由此可以使得爬虫的存活周期更长，也可以降低 IP 被封禁的风险。分页控制机制则可以实现精确控制和智能终止等功能，可以确保数据采集的完整性和效率，也可以优化计算机资源的使用。使用模块化的数据处理技术，使得数据质量有了保障，可以保证数据的完整性、准确性以及时效性，也可以提升爬虫的工作效率，降低维护的成本。

随着 Python 生态的持续发展，爬虫技术正变得越来越平台化、智能化。在使用爬虫技术的同时，更应该重视数据合规治理，构建可持续发展的数据采集体系。

2.3 分析算法介绍

1. 相关性分析是一种统计方法，通常用于研究两个或者多个变量之间的关联程度。它通过计算相关系数来衡量各变量之间线性关系的强弱。相关系数的取值范围在-1 到 1 之间，如果相关系数为 1，则表示完全正相关，为-1，如果相关系数为表示完全负相关，如果为 0，则表示无线性相关。相关性分析只能表明变量之间存在着一定的关联，但不能确定其因果关系。

2. 梯度提升算法是一种基于集成学习模型的机器学习算法，在数据挖掘与机器学习领域中占据重要地位。集成学习的核心思想是通过构建并结合多个学习器来完成学习任务，而梯度提升算法便是其中极具代表性的一种方法。

它通过迭代地训练一系列的弱学习器，在每一轮的迭代中，算法会基于当前模型的预测误差，训练一个新的弱学习器，随着迭代次数的增加，众多弱学习器最终融合为一个强大的预测模型。 这种训练方式使得梯度提升算法在预测性能上表现较好，能够提升模型的预测的准确度，为决策提供有力的支持。

梯度提升算法能够有效地捕捉复杂关系，具备较强的非线性建模能力和适应性，常被用来处理比较复杂的非线性关系问题。

3. K-means 聚类算法是一种无监督学习的聚类算法，它基于数据自身的特征和分布规律，将数据集分为了 K 个互不重叠的簇，将 K 个数据点作为初始质心，随后计算每个数据点离质心的距离并将它分配到最近的簇，再重新计算每个簇的质心，重复这个步骤直到质心不再发生显著的变化为止，这样使得簇的质心更能反映其内部数据的整体特征。

K-means 算法的优势在于它的高效性与实用性。其算法逻辑简洁清晰，计算过程主要围绕着距离度量和均值计算展开，很适合规模比较大的数据集。

4. TF-IDF 是自然语言处理和信息检索中一种重要的统计方法，用来评估一个词在文档集合中的重要程度。结合了词频（TF）与逆文档频率（IDF）这两个关键指标，从不同角度对词语的重要性进行量化。

词频（TF）用于分析单个文档内部，它反映的是某个词在一篇文档中出现的频繁程度，用来衡量这个词在文档中的重要程度。逆文档频率（IDF）用来衡量某个词的重要性。常见词的 IDF 值就比较低，反之，比较罕见的词的 IDF 值就高。使用 TF-IDF 可以有效地对文本进行情感分析。

TF-IDF 既考虑了每个词在文档中的重要性，也考虑了其在文档中的出现频率。使用 TF-IDF 能够降低常见词的占比权重，例如“的”、“是”这种常见词，提高了关键词的重要性。比较适合用于较短的文本，也适合大规模的文本任务。

5. SVM 是一种监督学习算法，主要作用于分类任务，它在文本数据中表现优秀。主要依赖于带有明确标签的训练数据进行模型训练，挖掘数据特征与标签之间的内在联系，在处理线性和非线性问题上都表现良好。

SVM 有许多优点：非常适合高维数据，支持关键样本决定分类边界，不受到大部分的非关键样本影响，即使训练数据较少，也能得到较好的分类效果，线性核在文本分类任务中通常表现很好，计算效率很高，也能捕捉到比较复杂的特征关系。能够在市场调研、舆情分析、用户反馈评价等实际的应用中提供可靠的分析结果，为决策提供有力的支持。

6. LDA 是一种无监督的概率主题模型，无需依赖预先标注的标签信息，而是通过挖掘数据自身的概率分布规律，在文本数据中自动探寻潜在的语义结构，能够在缺乏人工标注的情况下，高效地处理大规模文本集合。

LDA 主要用于文本挖掘和自然语言处理，能够从大量的文档中发现隐藏的主题，常常被用来辅助情感分析。LDA 可用于文本摘要提取，通过识别文档中的关键主题，提炼出最具代表性的内容。

第 3 章 数据采集

3.1 数据源

本研究所获取的数据主要是主要是来源于当当网，对当当网 Python 语言类图书的图书以及评论数据进行爬取和分析，当当网的图书信息相对比较稳定全面，涵盖市场上绝大多数图书信息，图书资源非常丰富，当当网是作为图书销售起家的平台，在图书领域有着深厚的积累，与众多出版社保持着长期合作的关系，其涵盖范围也比较广泛，包括各种类型、年代的图书，可以满足大多数消费者的需求。

并且当当网也经常推出促销活动，保持着产品价格竞争力，它的用户规模相对较大，有着强大的品牌影响力，可以通过分析当当网的数据来为电商平台提供思路，同时也可以为用户对 Python 类图书的选择提供参考。

3.2 图书基础数据采集

这部分爬取了当当网 Python 语言类图书的基础数据，爬取了书名、作者、出版社、价格、评论总数、星级等等关键字段，采用了比较常见的网页数据抓取方法和处理技术。在整体架构上，采用的是面向对象设计，并且封装为 DDSpider 类，在初始化时接收搜索关键词 key 作为参数，使用 requests 库向网站发送 HTTP 请求并获取响应，pyquery 解析和提取所需要的数据内容，再利用 pandas 对数据进行处理。

其中核心功能的设计如下：

1. 请求设计部分：

1）用户代理轮换：使用了随机 User-Agent 列表从而防止被识别为爬虫，这样做可以模拟出真实的浏览器行为，避免被反爬机制拦截，能够提高爬虫的稳定性和成功率。

2）分页处理：构造分页 URL,设定爬取的最大页数为 6 页，这样做可以减轻服务器的压力，避免因为单次请求过多数据对目标服务器造成过大的压力。同时可以一边抓取数据，一边进行处理，不用等待所有数据都获取完成。这种处理方式适用于大部分的网站，能够显著提高爬虫的效率。

3）请求间隔：设定了随机 1-3 秒的延迟，避免请求过于频繁，模拟浏览速度，可以降低检测风险，避免 IP 封禁。提高了爬虫的稳定性，也可以使得数据的质量有所保障。

4）数据解析：返回 DataFrame 格式的数据，并且判断 DataFrame 是否为空来决定是否继续爬取，这种模式适合分页爬取的场景。DataFrame 提供标准化的表格

结构，Pandas 会自动检测每列的数据类型，可以减少手动类型转换，并且 DataFrame 提供 isnull()、notna()等方法专门处理空值，使数据整理更加方便。当数据为空时跳过后续处理步骤，更加节省计算资源。DataFrame 可直接用于 Pandas 统计分析或机器学习，使得后续处理更加灵活简单。支持 to_csv/to_excel/to_sql 等多种储存方式，同时也可直接用于 Matplotlib/Seaborn 等可视化库。通过 DataFrame 的空值判断，实现了爬取流程的自我决策能力，这大大提升了爬虫的实用性。

5）数据存储：最终将每页数据暂存在 all_data 列表中，使用 pandas 合并所有数据，通过 save_to_csv 的方法保存。使用临时列表暂存并最终合并保存数据的好处和作用。这样可以对每页数据进行即时清洗和过滤，减少最终数据的体积，适合大数据量的爬取，同时可以完成断点恢复，在崩溃时可以从最后的成功页继续，已收集数据保留在 all_data 中。CSV 文件包含所有数据，方便后续分析使用，更易于管理和备份，这在数据完整性、效率和便利性之间取得了很好的平衡。

2. 获取数据以及解析数据部分：首先进行 URL 处理和 API 请求′product_id = url.split('/')[-1].split('.')[0]′从商品 URL 中提取 ID，随后 ′ api_url = f"https://product.dangdang.com/index.php?r = callback/detail&productId = {product_id}&..."′构造 API 以请求 URL，通过当当网的 API 接口来获取所需要的商品的 JSON 数据，这样能更加结构化。随后进行响应处理，′ response = requests.get(url, headers = headers, timeout = 10)′用来请求原始页面，′ api_response = requests.get (api_url, headers = headers, timeout = 10)′用来请求 API，这样设计可以从 API 优先获取数据，如果获取失败，则退回到解析 HTML。′ detected_encoding = chardet.detect (response.content)['encoding']html=response.content.decode(encoding, errors= 'replace')′用来处理编码问题，可以实现处理编码问题的功能，能够解决乱码，自动检测页面编码，避免出现解码错误的情况。′ api_data = api_response.json()result= extract_from_api (api_data, html)′尝试解析响应为 JSON，若成功，则调用 extract_from_api 函数；若 JSON 解析失败，则回退 HTML 解析。当 API 数据缺失时，使用 HTML 数据进行覆盖。具体代码详见代码清单 3-1：

代码清单 3-1

```
    product_id = url.split('/')[-1].split('.')[0]
        api_url =
f"https://product.dangdang.com/index.php?r=callback/detail&productId={product_id}&
templateType=publish&describeMap=0&shopId=0&categoryPath=01.03.57.05.00.00"
        try:
            response = requests.get(url, headers=headers, timeout=10)
            response.raise_for_status()
```

```
            detected_encoding = chardet.detect(response.content)['encoding']
            encoding = detected_encoding if detected_encoding else 'utf-8'
            html = response.content.decode(encoding, errors='replace')
            api_response = requests.get(api_url, headers=headers, timeout=10)
            api_response.raise_for_status()
            try:
                api_data = api_response.json()
                result = extract_from_api(api_data, html)
                return result
            except json.JSONDecodeError:
                html_data = extract_from_html(html)
                return html_data
        except requests.RequestException as e:
            print(f"请求失败: {e}")
            return None
def extract_from_api(api_data, html):
        result = {
            'title': "未获取到书名",
            'subtitle': "",
            'author': "未知",
            'publisher': "未知",
            'publish_time': "未知",
            'original_price': "未知",
            'current_price': "未知",
            'discount': "未知",
            'isbn': "未知",
            'img_url': "未获取到图片",
            'description': "",
            'details': {}
        }
if 'data' in api_data and 'product' in api_data['data']:
            product = api_data['data']['product']
            result['title'] = product.get('productName', "未获取到书名")
            result['subtitle'] = product.get('subTitle', "")
```

```
    if result['title'] == "未获取到书名" or result['author'] == "未知":
        html_data = extract_from_html(html)
        for key, value in html_data.items():
            if result[key] in ["未知", "未获取到书名", "未获取到图片", ""]:
                result[key] = value
        for key, value in html_data['details'].items():
            if key not in result['details']:
                result['details'][key] = value
    return result
def extract_from_html(html):
    doc = pq(html)
    result = {
        'title': "未获取到书名",
        'subtitle': "",
        'author': "未知",
        'publisher': "未知",
        'publish_time': "未知",
        'original_price': "未知",
        'current_price': "未知",
        'discount': "未知",
        'isbn': "未知",
        'img_url': "未获取到图片",
        'description': "",
        'details': {}
    }
    title_selectors = [
        'div.name_info h1',
        'div.product_wrapper h1',
        'div.product_info h1',
        '#product_info h1',
        '.name_info>h1',
        'h1[title]'
    ]
```

3. 关键字段部分：使用了多重选择器机制，每个获取的字段（标题、作者等等）都定义了一组可能的 CSS 选择器，然后按优先级逐个尝试，直到找到有效的数据。这样可以防止页面结构变动而导致发生数据抓取失败的情况，通过遍历所有可能的标题选择器，最后找到第一个非空结果之后停止。

具体代码详见代码清单 3-2：

代码清单 3-2

```
for selector in title_selectors:
    title_element = doc(selector)
    if title_element and title_element.text().strip():
        result['title'] = title_element.text().strip()
        break
subtitle_selectors = [
    'div.name_info h2',
    'div.product_wrapper h2',
    'div.product_info h2',
    '#product_info h2',
    '.name_info>h2'
]
for selector in subtitle_selectors:
    subtitle_element = doc(selector)
    if subtitle_element and subtitle_element.text().strip():
        result['subtitle'] = subtitle_element.text().strip()
        Break#多重选择器
author_selectors = [
    'span.author_type a',
    'span.author a',
    'div.messbox_info span.t1',
    '#author a',
    '.messbox_info .author a'
]
for selector in author_selectors:
    author_element = doc(selector)
    if author_element and author_element.text().strip():
        result['author'] = author_element.text().strip()
```

```
                break
        publisher_selectors = [
            'span.publisher_info a',
            'div.messbox_info span.t1 + span + span a',
            '#publisher a',
            '.publisher_info a[dd_name="出版社"]'
        ]
        for selector in publisher_selectors:
            publisher_element = doc(selector)
            if publisher_element and publisher_element.text().strip():
                result['publisher'] = publisher_element.text().strip()
                break
        publish_selectors = [
            'span.publisher_info',
            'div.messbox_info span:contains("出版时间")'
        ]
        for selector in publish_selectors:
            element = doc(selector)
            if element and "出版时间" in element.text():
                match = re.search(r'出版时间[:：]\s*(\d{4}[年-]\d{1,2}[月]?)',
element.text())
                if match:
                    result['publish_time'] = match.group(1)
                    break
        price_selectors = [
            'div.original_price',
            'div.price .price_info',
            '#original-price'
        ]
        for selector in price_selectors:
            element = doc(selector)
            if element and ("定价" in element.text() or "¥" in element.text()):
                price_text = element.text()
                match = re.search(r'[定价:：]?\s*(¥?\d+\.\d+)', price_text)
```

```
            if match:
                result['original_price'] = match.group(1)
                break
    current_price_selectors = [
        'p.price_d',
        'div.price .d_price',
        '#dd-price'
    ]
    for selector in current_price_selectors:
        element = doc(selector)
        if element and element.text().strip():
            result['current_price'] = element.text().strip()
            break
    discount_selectors = [
        'div.discount',
        'div.price .price_discount',
        '#discount'
    ]
    for selector in discount_selectors:
        element = doc(selector)
        if element and "折扣" in element.text():
            match = re.search(r'[折扣:：]?\s*(\d+\.\d+折|\d+折)', element.text())
            if match:
                result['discount'] = match.group(1)
                Break
```

这里设计了一些关键字段（部分）的提取细节：

标题（Title）：覆盖不同页面模板的标题位置，最终回退到通用选择器 `h1[title]`，再使用 `text().strip()` 去除空白字符，避免抓取到空内容。

副标题（Subtitle）结构与标题是类似的。

作者（Author）：定位包含作者信息的 `<a>` 标签（如 `span.author a`）。并考虑了作者信息在不同区域的展示方式（如 `.messbox_info .author a`）。

出版社（Publisher）：使用层级关系精准定位（如 `span.t1 + span + span a` 这样来表示相邻兄弟节点），并且通过属性选择器 `a[dd_name="出版社"]` 能够使准确性更加完备。

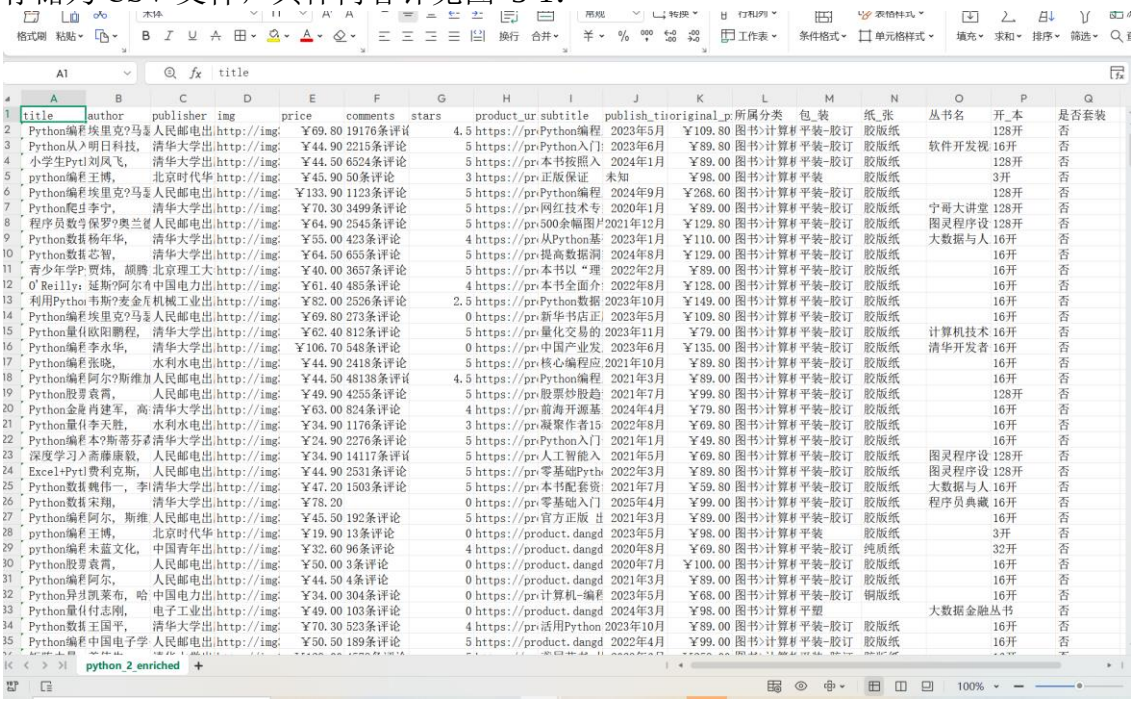
出版时间（Publish Time）：利用文本内容的过滤，查找包含“出版时间”文本的元素。并且利用正则表达式提取，从文本中匹配出版时间的日期格式（比如如 `2023 年 10 月` 或 `2023-10`）。

定价（Original Price）：利用文本关键词过滤，查找出来包含“定价”或“¥”的元素。再用正则表达式从中提取金额，匹配数字和小数点，能够对非结构化的文本进行精准的匹配，进行清洗以提高数据的质量。

当前价格（Current Price）：此处定位包含当前价格的元素（如促销价、折扣价），直接提取文本内容，通过 `break` 确保优先使用第一个有效结果。

折扣：进行文本关键词过滤，检查元素之中是否包含“折扣”二字。再正则匹配折扣的格式，比如 `6.5 折` 或 `7 折`。 `\d+\.\d+折`是匹配带小数点的折扣（如 `8.5 折`）。`\d+折`则是匹配整数折扣（如 `9 折`）。只有当格式完全匹配时才进行记录，避免错误数据。

最后再进行进度可视化和存储就完成了图书基础信息数据的爬取，最终结果存储为 CSV 文件，具体内容详见图 3-1：



	title	author	publisher	img	price	comments	stars	product_ur	subtitle	publish_ti	original_p	所属分类	包_装	纸_张	丛书名	开_本	是否套装
2	Python编程	埃里克?马瑟	人民邮电出	http://img	¥69.80	19176条评论	4.5	https://pr	Python编程	2023年5月	¥109.80	图书>计算机	平装-胶订	胶版纸		128开	否
3	Python从入	明日科技，	清华大学出	http://img	¥44.90	2215条评论	5	https://pr	Python入门	2023年6月	¥89.80	图书>计算机	平装-胶订	胶版纸	软件开发视	16开	否
4	小学生Pyt	刘凤飞，	清华大学出	http://img	¥44.50	6524条评论	5	https://pr	本书按照入	2024年1月	¥89.00	图书>计算机	平装-胶订	胶版纸		128开	否
5	python编程	王博，	北京时代华	http://img	¥45.90	50条评论	3	https://pr	正版保证	未知	¥98.00	图书>计算机	平装	胶版纸		3开	否
6	Python编程	埃里克?马瑟	人民邮电出	http://img	¥133.90	1123条评论	5	https://pr	Python编程	2024年9月	¥268.60	图书>计算机	平装-胶订	胶版纸		128开	否
7	Python爬虫	李宁，	清华大学出	http://img	¥70.30	3499条评论	5	https://pr	网红技术专	2020年1月	¥89.00	图书>计算机	平装-胶订	胶版纸	宁哥大讲堂	128开	否
8	程序员数学	保罗?奥兰德	人民邮电出	http://img	¥64.90	2545条评论	5	https://pr	500余幅图片	2021年12月	¥129.80	图书>计算机	平装-胶订	胶版纸	图灵程序设	128开	否
9	Python数据	杨年华，	清华大学出	http://img	¥55.00	423条评论	4	https://pr	从Python基	2023年1月	¥110.00	图书>计算机	平装-胶订	胶版纸	大数据与人	16开	否
10	Python数据	芯智，	清华大学出	http://img	¥64.50	655条评论	5	https://pr	提高数据洞	2024年8月	¥129.00	图书>计算机	平装-胶订	胶版纸		16开	否
11	青少年学P	贾炜，胡腾	北京理工大	http://img	¥40.00	3657条评论	5	https://pr	本书以“理	2022年2月	¥89.00	图书>计算机	平装-胶订	胶版纸		16开	否
12	O'Reilly:	延斯?阿尔布	中国电力出	http://img	¥61.40	485条评论	4	https://pr	本书全面介	2022年8月	¥128.00	图书>计算机	平装-胶订	胶版纸		16开	否
13	利用Pytho	韦斯?麦金尼	机械工业出	http://img	¥82.00	2526条评论	2.5	https://pr	Python数据	2023年10月	¥149.00	图书>计算机	平装-胶订	胶版纸		16开	否
14	Python编程	埃里克?马瑟	人民邮电出	http://img	¥69.80	273条评论	0	https://pr	新华书店正	2023年5月	¥109.80	图书>计算机	平装-胶订	胶版纸		16开	否
15	Python量化	欧阳鹏程，	清华大学出	http://img	¥62.40	812条评论	5	https://pr	量化交易的	2023年11月	¥79.00	图书>计算机	平装-胶订	胶版纸	计算机技术	16开	否
16	Python编程	李永华，	清华大学出	http://img	¥106.70	548条评论	0	https://pr	中国产业发	2023年6月	¥135.00	图书>计算机	平装-胶订	胶版纸	清华开发者	16开	否
17	Python编程	张晓，	水利水电出	http://img	¥44.90	2418条评论	5	https://pr	核心编程应	2021年10月	¥89.80	图书>计算机	平装-胶订	胶版纸		16开	否
18	Python编程	阿尔?斯维加	人民邮电出	http://img	¥44.50	48138条评论	4.5	https://pr	Python编程	2021年3月	¥89.00	图书>计算机	平装-胶订	胶版纸		16开	否
19	Python股	袁霄，	人民邮电出	http://img	¥49.90	4255条评论	5	https://pr	股票炒股趋	2021年7月	¥99.80	图书>计算机	平装-胶订	胶版纸		128开	否
20	Python金	肖建军，高	清华大学出	http://img	¥63.00	824条评论	4	https://pr	前海开源基	2024年4月	¥79.80	图书>计算机	平装-胶订	胶版纸		16开	否
21	Python量	李天胜，	水利水电出	http://img	¥34.90	1176条评论	3	https://pr	凝聚作者15	2022年8月	¥69.80	图书>计算机	平装-胶订	胶版纸		16开	否
22	Python编程	本?斯蒂芬森	清华大学出	http://img	¥24.90	2276条评论	5	https://pr	Python入门	2021年1月	¥49.80	图书>计算机	平装-胶订	胶版纸		16开	否
23	深度学习	斋藤康毅，	人民邮电出	http://img	¥34.90	14117条评论	5	https://pr	人工智能入	2021年5月	¥69.80	图书>计算机	平装-胶订	胶版纸	图灵程序设	128开	否
24	Excel+Pyt	费利克斯，	人民邮电出	http://img	¥44.90	2531条评论	5	https://pr	零基础Pyth	2022年3月	¥89.80	图书>计算机	平装-胶订	胶版纸	图灵程序设	128开	否
25	Python数	魏伟一，李	清华大学出	http://img	¥47.20	1503条评论	5	https://pr	本书配套资	2021年7月	¥59.80	图书>计算机	平装-胶订	胶版纸	大数据与人	16开	否
26	Python数	宋翔，	清华大学出	http://img	¥78.20		0	https://pr	零基础入门	2025年4月	¥99.00	图书>计算机	平装-胶订	胶版纸	程序员典藏	16开	否
27	Python编程	阿尔，斯维	人民邮电出	http://img	¥45.50	192条评论	5	https://pr	官方正版	2021年3月	¥89.00	图书>计算机	平装-胶订	胶版纸		16开	否
28	python编程	王博，	北京时代华	http://img	¥19.90	13条评论	0	https://product.dangd		2023年5月	¥98.00	图书>计算机	平装	胶版纸		3开	否
29	python编程	未蓝文化，	中国青年出	http://img	¥32.60	96条评论	4	https://product.dangd		2020年8月	¥69.80	图书>计算机	平装-胶订	纯质纸		32开	否
30	Python股	袁霄，	人民邮电出	http://img	¥50.00	3条评论	0	https://product.dangd		2020年7月	¥100.00	图书>计算机	平装-胶订	胶版纸		16开	否
31	Python编程	阿尔，	人民邮电出	http://img	¥44.50	4条评论	0	https://product.dangd		2021年3月	¥89.00	图书>计算机	平装-胶订	胶版纸		16开	否
32	Python异	凯莱布，哈	中国电力出	http://img	¥34.00	304条评论	0	https://pr	计算机-编程	2023年5月	¥68.00	图书>计算机	平装-胶订	铜版纸		16开	否
33	Python量	付志刚，	电子工业出	http://img	¥49.00	103条评论	0	https://product.dangd		2024年3月	¥98.00	图书>计算机	平塑		大数据金融丛书		否
34	Python数	王国平，	清华大学出	http://img	¥70.30	523条评论	4	https://pr	活用Python	2023年10月	¥89.00	图书>计算机	平装-胶订	胶版纸		16开	否
35	Python编	中国电子学	人民邮电出	http://img	¥50.50	189条评论	5	https://product.dangd		2022年4月	¥99.00	图书>计算机	平装-胶订	胶版纸		16开	否

图 3-1：图书基础数据图

3.3 用户评论数据爬取

这部分爬取了当当网 Python 语言类图书商品评论，提取的字段包含用户名、评分、评论内容、评论时间。其中使用了 requests 库向网站发送 HTTP 请求，再使用 BeautifulSoup 解析 HTML 内容、json 处理 JSON 数据，同时加入了 time 和 random 来用于控制请求的频率。

使用 requests 库向网站发送 HTTP 请求，作为爬虫的基础通信组件，负责与目标网站建立连接，来获取目标网站的响应，适用于不同网站的请求需求。利用 BeautifulSoup 解析 HTML 内容，可以支持多种解析器，适应不同的环境，可以精准地提取所需要的数据，处理不规范的 HTML 代码，容错性更强，同时也为后续的工作做准备。使用 json 处理 JSON 数据，可以解析返回的 JSON 格式数据。

使用 time 和 random 来控制请求频率，这样可以简单地实现请求间隔控制，也可以使得请求时间更加随机，用来模拟人类操作，这样可以有效地避免触发网站的反爬机制，使得爬虫能够更加稳定和安全。这些操作使得爬虫整体的功能更加完整，大大增强了代码的稳定性和扩展性，后续进行维护也非常方便。

数据爬取流程：

1. 设置目标评论数，控制工作范围，避免因为持续爬取数据所造成的资源浪费；

2. 使用 while 循环直到收集足够评论可以持续运行并且不需要人工干预；

3. 随机延迟 2-4 秒避免被封，这样可以控制请求频率，有效地降低了风险；

4. 从 JSON 中提取 HTML 评论内容，可以处理复杂的数据，增强代码的可靠性；

5. 使用 BeautifulSoup 解析 HTML 可以提取结构化信息，处理不规范的 HTML 代码；

6. 每爬取 5 页保存一次数据，这样分批去保存数据能够避免若因为意外中断而导致数据全部丢失的突发情况；

7. 每 5 页休息较长时间，避免反爬限制，使运行更加安全；

8. 保存数据，以便后续的分析和处理。

本部分基本上完整地展示了爬虫工作流程，可以实现自动化控制，基于预定数据量采用智能终止机制，通过动态延迟和周期性休息可以完成自适应，每爬取一定量的数据就进行自动保存，避免意外终止导致数据全部丢失这种情况。同时也建立了完备的保护机制，采用 2-4 秒的随即延迟，每爬取五页休息一段时间，可以有效地避免触发网站的反爬机制，增强稳定性和安全性。同时数据的质量也有所保障，先 JSON 提取再 HTML 解析，并且每爬取 5 页数据就进行独立保存，避免批量失败。

1. 获取评论数据部分

这部分的代码用于爬取当当网指定商品（productId=29564789）的用户评论数据，后续可以进入网站获取其他商品的 productId，改变相应的代码即可爬取相应的指定商品，目标是获取至少 3000 条评论。

首先进行了请求头设置，"Referer"可以标识请求来源页面，绕过简单的反爬，"Cookie" 包含了用户会话信息，用来进行权限验证，"User-Agent"用来伪装浏览器访问行为，避免被识别为爬虫触发网站反爬机制。下一部分是 URL 模板，这里 productId=29564789 是商品的唯一标识，categoryPath=01.54.06.19.00.00 指的是商品分类路径， pageIndexsh 分页参数。

具体代码详见代码清单 3-3：

代码清单 3-3

```
    headers = {
        "Referer": "https://product.dangdang.com/29564789.html",
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)""AppleWebKit
/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/ 537.36",
        "Cookie": "ddscreen=2;
        __permanent_id=20250313182614765200354815671957987;
        secret_key=4229ccff1bab6b5fa625d54dc58d5c47;
        sessionID=pc_88aaeb1d26c51a085764b25e1eec4a4f42086ce637ea8a5f25597
aa2584f9e8f;
        USERNUM=Mzd4DQGjGP4wx4GLSKI0Eg==;
        login.dangdang.com=.ASPXAUTH=JshDNEre8U57q2+uglNcLlDJv9vHDYd
4mxfZp4xtRNGgc+do+TvhRg==;
        dangdang.com=email=MTU2ODk1NjcyMjE2NDgyMUBkZG1vYmlscGhvb
mVfX3VzZXIuY29t&nickname=&display_id=5605830714260&customerid=B0f7yMn
EyYnZVHv064aabA==&viptype=eqrIvDSQWI8=&show_name=156****7221;
        pos_9_end=1741874639023;
        pos_0_end=1741874639219;
        pos_0_start=1741921698905;
        ad_ids=88943754%7C%231;
        dest_area=country_id%3D9000%26province_id%3D111%26city_id%20%3D
0%26district_id%3D0%26town_id%3D0;
        LOGIN_TIME=1741941907304;
        __visit_id=20250314164507312245698235605655552;
        __out_refer=;
        __trace_id=20250314164652660192598563267631325;
        pos_6_end=1741942012870;
        pos_6_start=1741942015700;
```

```
        __rpm=s_112100.94003212839%2C94003212840.1.1741922079914%7Cp_2
9520739.comment_body..1741942039022"
    }
    url_template="https://product.dangdang.com/index.php?r=comment%2Flist&prod
uctId=29564789&categoryPath=01.54.06.19.00.00&mainProductId=29564789&mediu
mId=0&pageIndex={}&sortType=1&filterType=1&isSystem=1"
    comments_list = []
    total_comments_target = 3000
    current_page =1
```

接下来这部分使用了主循环控制方法，每批爬取 5 页，持续地爬取数据直到评论数达到设定的目标。先发送 HTTP 请求，headers 包含了 Cookie 和 UA 的完整请求头，同时设置了 2-4 秒的请求间隔时间，有效地减轻了压力，避免触发网站的反爬机制。随后解析 JSON 响应、解析 HTML 评论数据，最后提取单条评论的信息，提取的字段包含用户名、评分、评论内容、评论时间。

具体代码详见代码清单 3-4：

代码清单 3-4

```
    while len(comments_list) < total_comments_target:
        for i in range(5):   # 每批爬取 5 页
            response = requests.get(url_template.format(current_page), headers =
headers)
            time.sleep(random.uniform(2, 4))   # 每次请求间隔 2-4 秒
            try:
                data = response.json()
                #print(response.text)
                html = data.get("data", {}).get("list", {}).get("html", "")
                soup = BeautifulSoup(html, 'html.parser')
                comment_items = soup.select(".comment_items")
                if not comment_items:
                    print(f"第{current_page}页没有评论数据，结束爬取。")
                    break
                for comment_item in comment_items:
                    try:
                      user = comment_item.select_one(".items_left_pic.name").
text.strip()
```

```
                    rating = comment_item.select_one(".pinglunem").text.
strip()
                    review_text = comment_item.select_one(".describe_detail
span").text.strip()
                    comment_time = comment_item.select_one(".starline
span").text.strip()
                    comments_list.append({
                        "用户名": user,
                        "评分": rating,
                        "评论内容": review_text,
                        "评论时间": comment_time
                    })
```

2. 反爬与存储部分

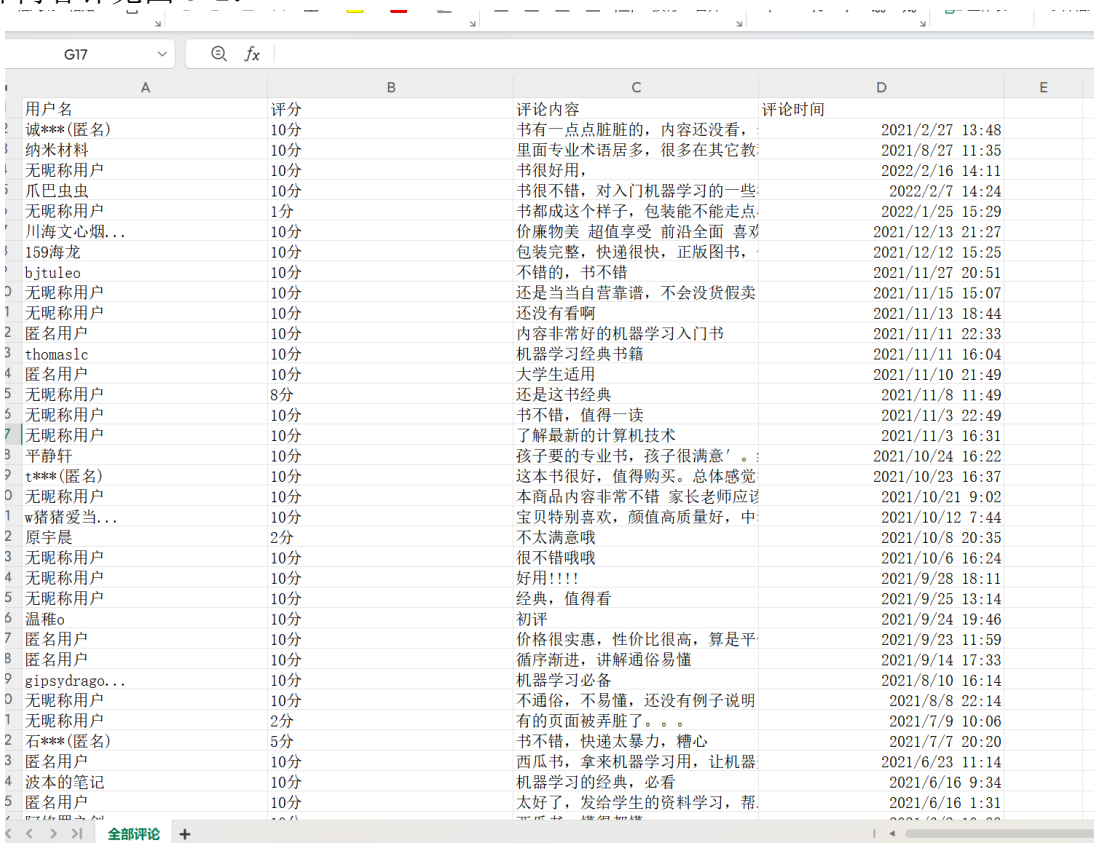
这部分的代码实现了分页数据保存和反爬策略，采用分页保存机制,每爬取五页就保存一次已爬取的数据信息，这样可以避免因意外中断程序而导致全部数据丢失的情况。page_{current_page - 1} 标识当前批次最后一页（例如爬完第 2 页时保存为 page_2.csv），这样设计实现了文件名动态化。utf-8-sig 解决了 Excel 打开 CSV 文件时的中文乱码问题，代码中也实现了明确进度的这一功能。每爬取五页数据后就休息一段时间，模拟了人类的浏览行为，也可以再关键节点暂停并检查 Cookie 有效性，这样的设计可以有效避免被网站反爬限制。具体代码详见代码清单 3-5：

代码清单 3-5

```
    # 每爬取 5 页保存一次数据
        df = pd.DataFrame(comments_list)
        df.to_csv(f"dangdang_comments_page_{current_page - 1}.csv", index=False,
encoding='utf-8-sig')
        print(f"\n 已保存数据到 dangdang_comments_page_{current_page -
1}.csv，当前进度到第{current_page - 1}页，共爬取{len(comments_list)}条评论。
\n")#
    # 每 5 页休息较长时间，避免反爬限制
        print("休息 3 秒，避免被网站限制...")
        time.sleep(3)
    # 提示检查 Cookie 有效性，避免频繁失败
```

```
    #input("请检查 Cookie 是否仍有效，确认后按回车继续，或 Ctrl+C 终止爬取...")
    print("所有数据已成功爬取完毕！")
    # 最终数据保存
    df = pd.DataFrame(comments_list)
    df.to_csv("dangdang_comments_final_test.csv", index=False, encoding='utf-8-sig')
    print("数据已保存至 dangdang_comments_final.csv")
```

最后再进行数据保存，将数据保存至 CSV 文件当中，方便后续分析使用。具体内容详见图 3-2：



用户名	评分	评论内容	评论时间
诚***(匿名)	10分	书有一点点脏脏的，内容还没看，	2021/2/27 13:48
纳米材料	10分	里面专业术语居多，很多在其它教	2021/8/27 11:35
无昵称用户	10分	书很好用，	2022/2/16 14:11
爪巴虫虫	10分	书很不错，对入门机器学习的一些	2022/2/7 14:24
无昵称用户	1分	书都成这个样子，包装能不能走点	2022/1/25 15:29
川海文心烟...	10分	价廉物美 超值享受 前沿全面 喜欢	2021/12/13 21:27
159海龙	10分	包装完整，快递很快，正版图书，	2021/12/12 15:25
bjtuleo	10分	不错的，书不错	2021/11/27 20:51
无昵称用户	10分	还是当当自营靠谱，不会没货假卖	2021/11/15 15:07
无昵称用户	10分	还没有看啊	2021/11/13 18:44
匿名用户	10分	内容非常好的机器学习入门书	2021/11/11 22:33
thomaslc	10分	机器学习经典书籍	2021/11/11 16:04
匿名用户	10分	大学生适用	2021/11/10 21:49
无昵称用户	8分	还是这书经典	2021/11/8 11:49
无昵称用户	10分	书不错，值得一读	2021/11/3 22:49
无昵称用户	10分	了解最新的计算机技术	2021/11/3 16:31
平静轩	10分	孩子要的专业书，孩子很满意'。	2021/10/24 16:22
t***(匿名)	10分	这本书很好，值得购买。总体感觉	2021/10/23 16:37
无昵称用户	10分	本商品内容非常不错 家长老师应该	2021/10/21 9:02
w猪猪爱当...	10分	宝贝特别喜欢，颜值高质量好，中	2021/10/12 7:44
原宇晨	2分	不太满意哦	2021/10/8 20:35
无昵称用户	10分	很不错哦哦	2021/10/6 16:24
无昵称用户	10分	好用!!!!	2021/9/28 18:11
无昵称用户	10分	经典，值得看	2021/9/25 13:14
温稚o	10分	初评	2021/9/24 19:46
匿名用户	10分	价格很实惠，性价比很高，算是平	2021/9/23 11:59
匿名用户	10分	循序渐进，讲解通俗易懂	2021/9/14 17:33
gipsydrago...	10分	机器学习必备	2021/8/10 16:14
无昵称用户	10分	不通俗，不易懂，还没有例子说明	2021/8/8 22:14
无昵称用户	2分	有的页面被弄脏了。。。	2021/7/9 10:06
石***(匿名)	5分	书不错，快递太暴力，糟心	2021/7/7 20:20
匿名用户	10分	西瓜书，拿来机器学习用，让机器	2021/6/23 11:14
波本的笔记	10分	机器学习的经典，必看	2021/6/16 9:34
匿名用户	10分	太好了，发给学生的资料学习，帮	2021/6/16 1:31

图 3-2：评论信息数据图

3.4 数据预处理

1.出版社数据预处理

从 CSV 文件中加载数据，删除了包含空值的行，并创建保存结果的目录，再对价格数据进行了清洗，这里移除了货币符号和逗号，然后从评论字段中提取评论数量。

具体代码详见代码清单 3-6：

代码清单 3-6

```
df = pd.read_csv('ok_Python_book_data.csv')
df = df.dropna(subset=['title', 'author', 'publisher', 'price', 'comments', 'stars'])
print(f"数据加载完成，共 {len(df)} 条记录")
# 创建结果目录
result_dir = 'Python 图书数据分析结果'
if not os.path.exists(result_dir):
    os.makedirs(result_dir)
    print(f"创建结果目录: {result_dir}")
else:
    print(f"结果目录已存在: {result_dir}")
# 清洗数据
df['price'] = df['price'].replace({'¥': '', ',': ''}, regex=True).astype(float)
df['comments_count'] = df['comments'].str.extract('(\d+)').astype(float)
```

2.图书销量数据预处理

对图书数据进行了清洗，提取了评论数量并将其转换为浮点数，同时将价格数据和评论数据也进行了清洗，这部分移除了货币符号和千位分隔符，将价格字段统一为数值格式，移除了“评分”列中的“分”字并转换为五分制，并且填充了缺失的星级数据。

清洗了作者的数据：处理了缺失值，移除了括号里的内容，例如国籍等信息，如果有多个作者，则进行分割并只取第一个作者。具体代码详见代码清单 3-7。

代码清单 3-7

```
# 清洗图书数据
df_books = df_books.dropna(subset=['title', 'author', 'publisher', 'price'])
# 提取评论数量
df_books['comments_count'] = df_books['comments'].str. extract('(\d+)'). Astype
(float)
# 清洗价格数据
df_books['price'] = df_books['price'].replace({'¥': '', ',': ''},regex=True). Astype
(float)
# 填充缺失的星级数据
df_books['stars'] = df_books['stars'].fillna(0)
# 清洗评论数据
df_comments['评分'] = df_comments['评分'].str.replace('分', '').astype(float) / 2
```

```
# 转换为 5 分制
def clean_author(author):
    if pd.isna(author):
        return '未知'
    # 移除括号内的内容，如"(美)"等
    author = re.sub(r'\([^)]*\)', '', author)
    # 分割多个作者，只取第一个
    if ',' in author:
        author = author.split(',')[0].strip()
    return author.strip()
```

3.热销图书用户情感数据预处理

加载了停用词文件，如果文件不存在就使用默认的停用词，并且使用 jieba 进行中文分词，过滤停用词和单字词，返回用空格连接的词串，并且对评分数据进行了清洗，去掉“分”字并且转化为浮点数，对于每一条评论都进行了分词和停用词过滤。

具体代码详见代码清单 3-8：

代码清单 3-8

```
# 停用词
def load_stopwords(file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            stopwords = f.readlines()
        return set([word.strip() for word in stopwords])
    except FileNotFoundError:
        print(f"停用词文件 {file_path} 不存在，将使用默认停用词")
        return set(['的', '了', '和', '是', '就', '都', '而', '及', '与', '着', '或'])
# 分词和去除停用词
def jieba_cut(text, stopwords):
    words = jieba.cut(text)
    return " ".join([word for word in words if word not in stopwords and
len(word) > 1])
# 创建结果目录
result_dir = 'Python 图书评论情感分析结果'
if not os.path.exists(result_dir):
```

```
    os.makedirs(result_dir)
# 加载数据
data = pd.read_csv('全部评论.csv', encoding='utf-8-sig')
data['评分'] = data['评分'].str.replace('分', '').astype(float)
stopwords = load_stopwords('stopword.txt')
# 对评论内容进行分词和去停用词
data['评论内容'] = data['评论内容'].apply(lambda x: jieba_cut(str(x), stopwords))
```

第 4 章 数据分析与可视化实现

4.1 出版社数据分析与可视化

本模块对于出版社的数据进行了详细的研究，主要研究了不同出版社的读者满意度、不同出版社的图书定价、不同出版社的图书销量以及不同出版社的图书出版量，并且根据这些数据对部分特征值进行了相关性分析。通过分析这些指标的相关性，可以揭示更深层的市场关系，可以帮助出版社优化自己的营销策略，也为读者选择出版社或图书提供较有参考价值的依据。

4.1.1 出版社读者满意度分析

对出版社进行平均星级分析能够直观地反映出来该出版社的总体质量以及消费者的满意度，可以识别出哪家出版社的 Python 类图书总体质量较高。

在出版社平均星级分析中，首先按照出版社分组计算平均星级并且进行降序排列，选取其中的前 10 名。使用 seaborn.barplot 来绘制条形图展示结果，设置出版社名称旋转 90 度防止重叠，标题为“Top 10 出版社的平均星级评分”，x 轴为“出版社”，y 轴为“平均星数”，`tight_layout()`则可以自动调整子图参数，这样做可以避免标签被截断。

具体代码详见代码清单 4-1：

代码清单 4-1

```
publisher_avg_stars = df.groupby('publisher')['stars'].mean().sort_values
(ascending = False)
top_10_publishers_stars = publisher_avg_stars.head(10)
plt.figure(figsize = (12, 6))
sns.barplot(x = top_10_publishers_stars.index, y = top_10_publishers_stars.values)
plt.xticks(rotation = 90)
plt.title('Top 10 出版社的平均星级评分')
plt.xlabel('出版社')
plt.ylabel('平均星级')
plt.tight_layout()
plt.savefig(f'{result_dir}/出版社平均星级评分 Top10.png', dpi = 300)
plt.close()
```

具体内容详见图 4-1：

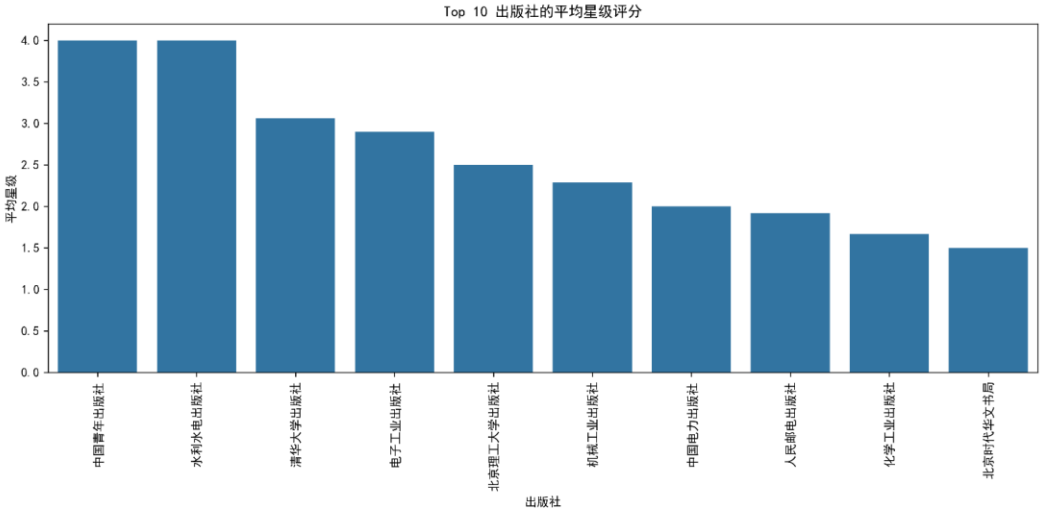


图 4-1：Top 10 出版社的平均星级图

见图 4-1，在 Python 语言类图书这一图书种类中，不同出版社的平均星级按由高到低的前十名分别是中国青年出版社、水利水电出版社、清华大学出版社、电子工业出版社、北京理工大学出版社、机械工业出版社、中国电力出版社、人民邮电出版社、化学工业出版社、北京时代华文书局。其中中国青年出版社和水利水电出版社这两个出版社的平均星级遥遥领先，基本都达到了四星，可以看出消费者对这两家出版社的图书比较满意，剩下的八家出版社的平均星级保持稳定的递减，北京时代华文书局的平均星级是最低的，仅仅只有一点五星左右，需要反思并改进自己的问题。

4.1.2 出版社定价分析

对不同出版社的平均价格进行分析可以体现出该出版社 Python 类图书的市场定位，例如是高端或者是平价方向，高价出版社可能更多专注于专业领域，而比较平价的出版社可能专注于入门教材等方向。分析平均价格可以帮助消费者更快地选择产品，对于出版社来说，可以帮助竞争对手调整定价策略。

这里的步骤与星际分析大概类似，先计算出各个出版社的平均价格并且取其中的前十名，然后再绘制前十名的出版社的价格条形统计图：设置 x 轴标签旋转 90 度防止重叠，标题设置为“Top 10 出版社平均价格排名”，x 轴为“出版社”，y 轴为“平均价格”，`tight_layout()`用来自动调整子图参数。

具体代码详见代码清单 4-2：

代码清单 4-2

```
publisher_avg_price = df.groupby('publisher')['price'].mean().sort_values
(ascending=False).head(10)
```

```
plt.figure(figsize=(12, 6))
sns.barplot(x=publisher_avg_price.index, y=publisher_avg_price.values)
plt.xticks(rotation=90)
plt.title('Top 10  出版社平均价格排名')
plt.xlabel('出版社')
plt.ylabel('平均价格  (元)')
plt.tight_layout()
plt.savefig(f'{result_dir}/出版社平均价格 Top10.png', dpi=300)
plt.close()
```

具体内容详见图 4-2：

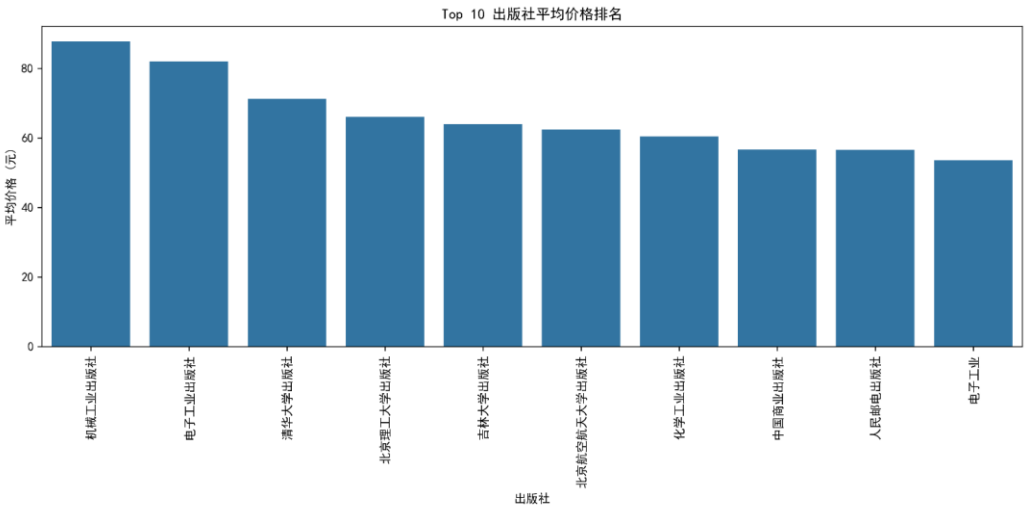


图 4-2：Top 10 出版社平均价格排名图

见图 4-2，在 Python 语言类图书这一图书种类中，不同出版社的价格排名分别是机械工业出版社、电子工业出版社、清华大学出版社、北京理工大学出版社、吉林大学出版社、北京航空航天大学出版社、化学工业出版社、中国商业出版社、人民邮电出版社、电子工业。与之前的平均星级分析图并不重合，说明影响消费者满意度的因素不仅仅有价格，并不是价格越低消费者满意度就越高，还要考虑图书的质量品控问题、商家的营销策略等等方面，才能更好地服务用户。

4.1.3 出版社图书销量分析

对出版社评论总数进行分析能够反映出该出版社的市场热度，评论总数通常都与销量有着正相关的关系，能够识别出市场占有率比较高的出版社。

首先从 comments 列提取数字评论数，然后计算各出版社评论总数并降序排列并绘制出条形图展示结果，设置 x 轴标签旋转 90 度防止重叠，标题设置为“Top 10 出版社的评论总数”，x 轴为“出版社”，y 轴为“评论总数”。

具体代码详见代码清单 4-3：

代码清单 4-3

```
publisher_comments = df.groupby('publisher')['comments_count'].sum().
sort_values(ascending=False)
top_10_publishers_comments = publisher_comments.head(10)
plt.figure(figsize=(12, 6))
sns.barplot(x = top_10_publishers_comments.index, y = top_10_publishers_
comments.values)
plt.xticks(rotation=90)
plt.title('Top 10  出版社的评论总数')
plt.xlabel('出版社')
plt.ylabel('评论总数')
plt.tight_layout()
plt.savefig(f'{result_dir}/出版社评论总数 Top10.png', dpi=300)
plt.close()
```

具体内容详见图 4-3：

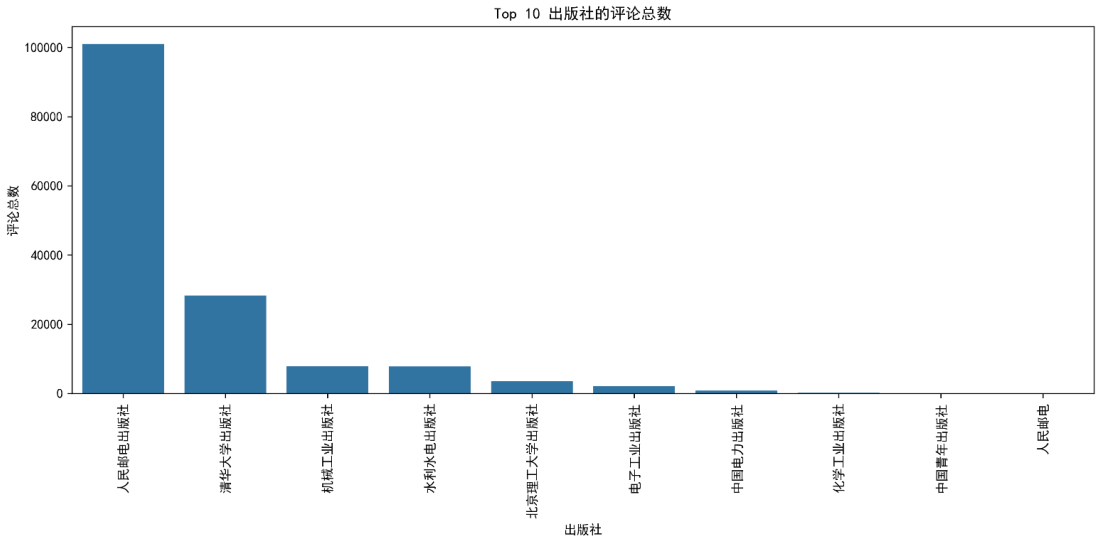


图 4-3：Top 10 出版社的评论总数图

根据对各个出版社评论数量的研究分析，可以很明显地看出人民邮电出版社是所有出版社中评论数量最多、占比最大的出版社，达到了万条以上，结合价格排名，人民邮电出版社也是定价较低的出版社之一，消费者的反馈与价格之间有着直接的密不可分的关联，其余的九家出版社仅仅占到了大概评论总数的三分之一，仅清华大学出版社占比稍多一些。

4.1.4 出版社图书出版量分析

分析出版社的 Python 图书出版量能具体地展示出该出版社在 Python 领域的投入程度，出版数量越多的出版社，往往是在计算机技术图书领域的头部出版社，或者是更专注于技术图书的出版社。

这里统计了每家出版社出版的 Python 类图书的数量，并且按照降序的顺序取前 10 名，直接统计每个分组中的记录数，x 轴为“出版社”，y 轴为“图书数量”绘制“Top 10 出版社 Python 图书数量”。

具体代码详见代码清单 4-4：

代码清单 4-4

```
publisher_book_count = df.groupby('publisher').size().sort_values
(ascending=False).head(10)
plt.figure(figsize=(12, 6))
sns.barplot(x=publisher_book_count.index, y=publisher_book_count.values)
plt.xticks(rotation=90)
plt.title('Top 10 出版社 Python 图书数量')
plt.xlabel('出版社')
plt.ylabel('图书数量')
plt.tight_layout()
plt.savefig(f'{result_dir}/出版社图书数量 Top10.png', dpi=300)
plt.close()
```

具体内容详见图 4-4：

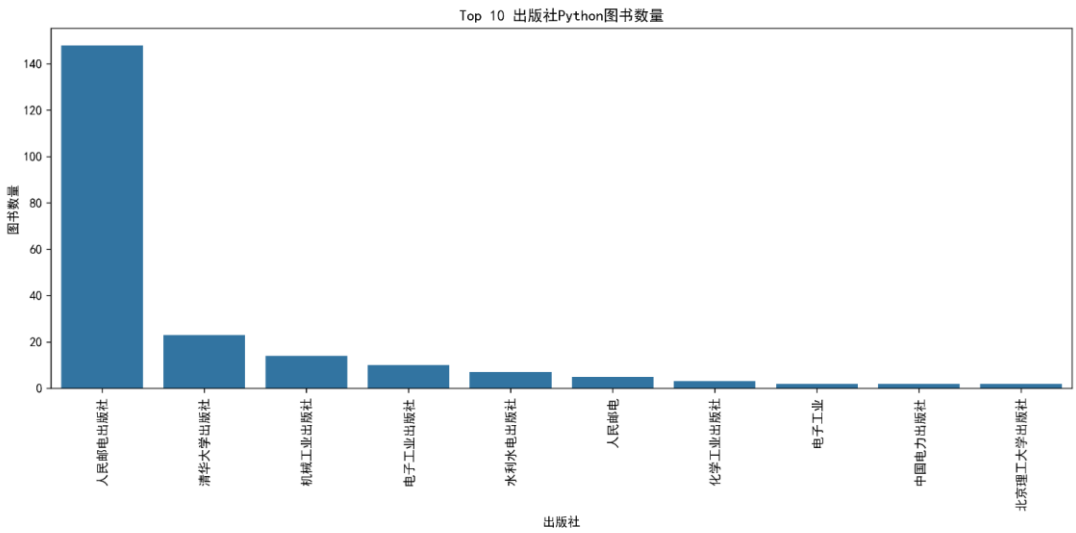


图 4-4：Top 10 出版社 Python 图书数量图

见图 4-4，各个出版社出版 Python 类图书数量由高到低依次是人民邮电出版社、清华大学出版社、机械工业出版社、电子工业出版社、水利水电出版社、人民邮电、化学工业出版社、电子工业、中国电力出版社、北京理工大学出版社。其中出版最多的是人民邮电出版社，其他出版社的数量水平都比较平均，可以看出人民邮电大学是在计算机技术图书领域的头部出版社。

4.1.5 相关性分析

研究了价格、评论数量与星级评分之间的相关性，由于并没有可以直接使用的销量数据，在此处利用评论数量来近似地代替销量。评论数和销量在一定程度上反映了产品的市场受欢迎程度，星级评分则体现了消费者对产品的认可程度。分析它们之间的相关性，能够让商家更全面地评估产品的市场表现。消费者在购买产品时，往往会参考价格、销量和其他消费者的评价。相关性分析结果可以帮助消费者更直观地了解这些因素之间的关系，从而做出更明智的购买决策。这样研究能够了解到各个变量之间更深层的关系，能够在一定程度上为平台、电商以及用户提供决策依据。可以通过对结果的分析，优化对这三者的组合，从而实现利益最大化。

在相关性分析中使用了热图展示了 Spearson 相关系数矩阵，对于异常值和衡量非线性关系更加稳定，主要使用了 pandas 库用于数据处理，并且还使用了 matplotlib 库 + seaborn 库进行数据可视化处理，指定使用 Spearman 秩相关系数计算，直接在热力图每个格子中显示相关系数数值，便于快速读取。

具体代码详见代码清单 4-5：

代码清单 4-5

```
# 计算 Spearman 相关系数（非参数，对异常值更稳健）
spearman_corr = correlation_data.corr(method='spearman')
plt.figure(figsize=(8, 6))
sns.heatmap(spearman_corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('价格、评论数量与星级评分的 Spearman 相关性矩阵')
plt.tight_layout()
plt.savefig(f'{result_dir}/Spearman 相关性矩阵.png', dpi=300)
plt.close()
```

所得 Spearman 相关性矩阵详见图 4-5：

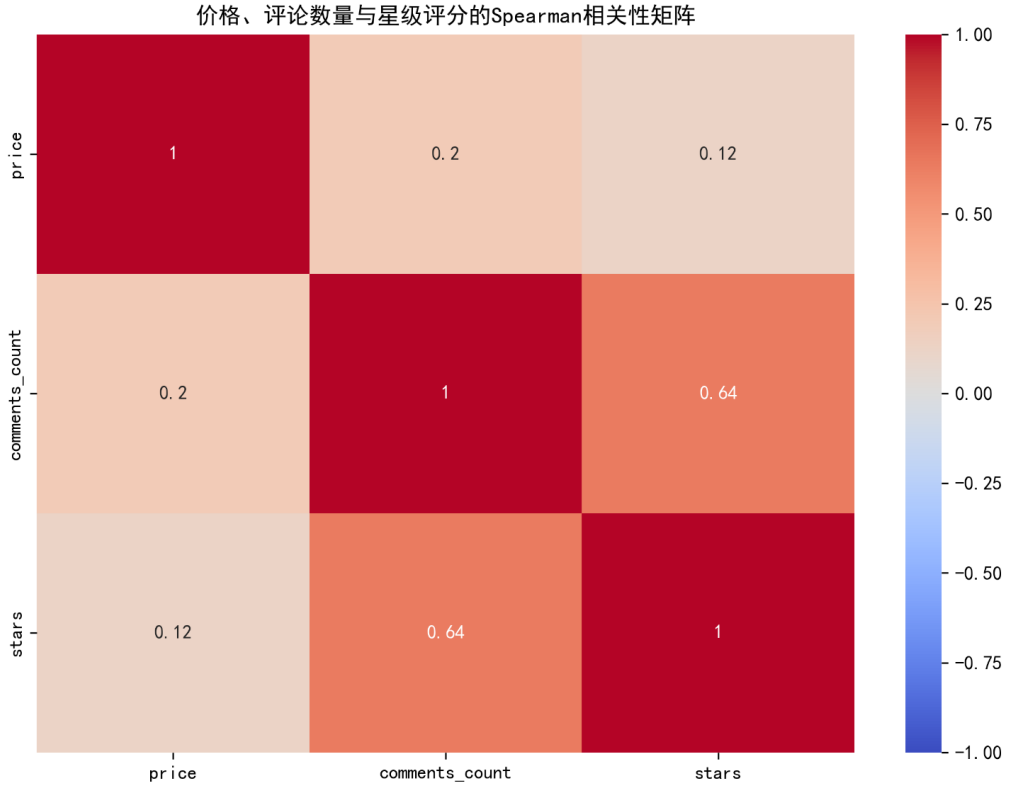


图 4-5：价格、评论数量与星级评分的 Spearman 相关性矩阵

矩阵图中的每个单元格都表示两个变量之间的相关性系数，为了更直观地展示相关性的强弱，矩阵图通常会用颜色来表示相关性系数的大小，红色表示正相关，颜色越深，相关性越强；蓝色表示负相关，同样颜色越深，负相关性越强。

见图 4-5，价格与评论数量的相关性为 0.2，而价格与星级评分的相关性为 0.12，评论数量与星级评分相关性为 0.64，可以看出评论数量与星级评分间相关性最大，平台和商家应该多关注评论数量与星级评分之间的联系，这代表了销量与用户满意度之间的关系，应当制定更好的营销策略和定价思路，适时地进行一些促销活动或者评论返现活动，使图书的评论数与其评分能够进行正向的循环，从而带动图书的销量。

鉴于评论数量与星级相关性较高，对于该特征值绘制散点图分析，具体内容详见图 4-6：

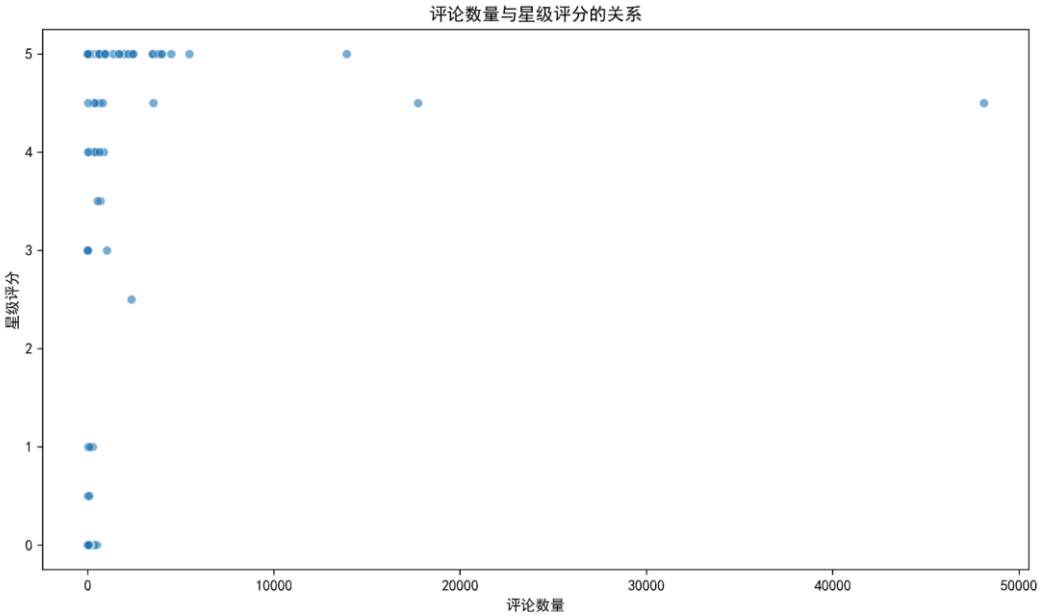


图 4-6：评论数量与星级评分的关系图

横轴代表评论数量，纵轴代表星级评分，可以通过观察散点的分布状况来了解评论数块与星级评分之间的关系。

评论数量越多时星级评分往往比较高，销量与用户满意度二者呈现出来正相关的关系，有的图书拥有着好的口碑循环，好评驱动了更多的购买和评价，可以更多地关注高星级但评论不是很多的图书，这种图书的潜力较大，通过促销快速地提高评论数量，形成正向的循环。

4.2 图书销量数据分析

本部分的研究对高产作者、图书的应用领域以及图书销量的相关性，能够帮助了解哪些因素会影响图书的销量，揭示它们之间的潜在关系，为出版社的选择合作对象、选题等方面提供一些依据。

并且对图书的销量进行了预测，在这部分研究中评估了五种模型，最终根据 R^2 值选择梯度提升为最佳模型。可以帮助出版社确定合理的库存并制定营销计划，有助于出版社在竞争激烈的市场当中找到适合自身发展的出版策略。

4.2.1 作者分析

通过对作者的分析，可以识别出市场上高产的 Python 技术作者，为出版社寻找到潜在的合作作者，也可以了解到哪些作者的图书销量最高，或者哪些高产作者的图书的读者满意度最高，可以使平台和出版社有效地把握市场动向，了解市场竞

争格局。对于读者来说，通过分析结果能够有效地识别出高产高质的作者，提高购书的质量，具有一定的参考价值。

1. 作者作品数量统计与高产作者作品信息提取

通过统计作者作品数量并提取作品信息，能够量化作者的作品产出能力，直接反映作者在 Python 图书领域的活跃度及输出能力。通过系统地统计作者作品数量，并深入提取作品的详细信息，构建一套科学、量化的评估体系，精准衡量作者在 Python 图书领域的创作表现。

核心逻辑主要分为两步：统计作者作品数量和提取高产作者的作品详情，对 df_books 数据框中 clean_author 列（作者名）进行计数，返回每个作者的作品数量，author_counts >= 2 过滤出作品数量≥2 的作者，排除仅发表 1 部作品的作者。随后循环遍历高产作者，筛选作者对应的作品数据。该部分代码详见代码清单 4-6：

代码清单 4-6

```
# 统计每个作者的作品数量
author_counts = df_books['clean_author'].value_counts()
top_authors = author_counts[author_counts >= 2].reset_index()
top_authors.columns = ['author', 'book_count']

# 获取每个高产作者的作品信息
top_author_books = {}
for author in top_authors['author']:
    author_df = df_books[df_books['clean_author'] == author].copy()
    author_df['short_title'] = author_df['title'].apply(lambda x: x[:20] + '...' if
len(x) > 20 else x)
    top_author_books[author] = author_df[['short_title', 'price', 'comments_count',
'stars']]
```

2.高产作者产量分析

将数值转换为直观的长度对比，能够明显地感受到作者之间作品产量的差距。高产作者通常在该领域投入了更多时间和精力，持续输出作品，展现出较高的创作热情和积极性。出版机构可以通过分析作者产量，识别出具有市场潜力和创作实力的作者，从而更有针对性地与他们合作，提高出版效率和市场竞争力。具体内容详见图 4-7：

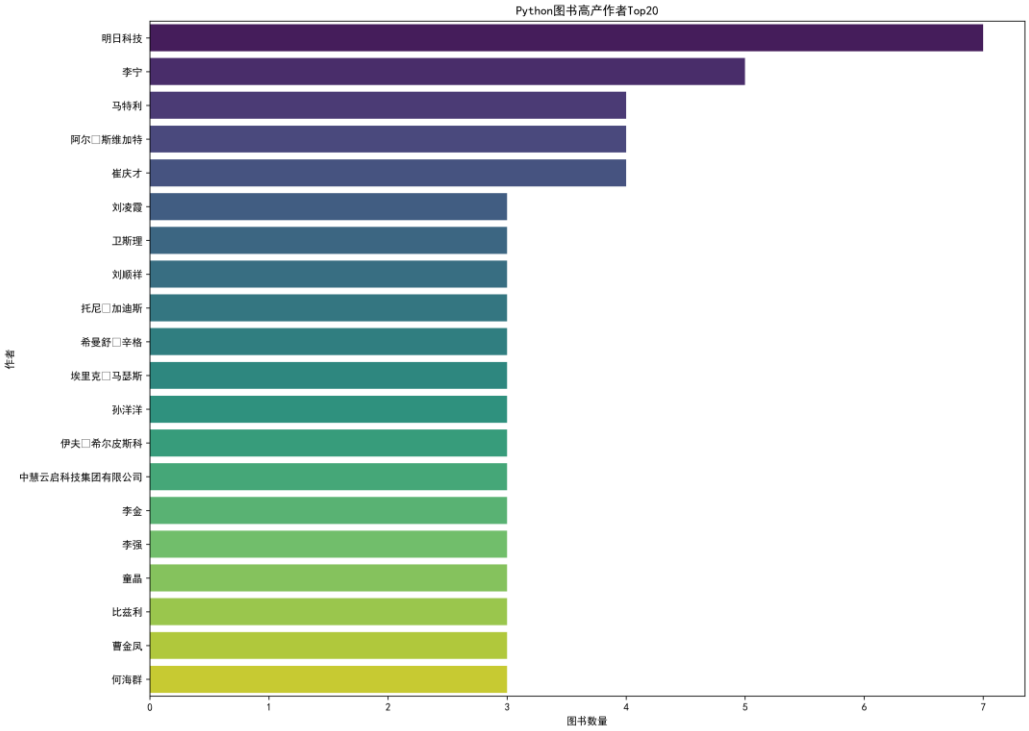


图 4-7：Python 图书高产作者 Top20

见图 4-7，横轴为图书数量，纵轴为作者。明日科技的 Python 类图书数量产出最多，随后是李宁、马特利、阿尔斯维加特、崔庆才等个人作家，机构作者的高产反映出其具有团队化创作模式或者标准化的内容生产流程，个人作者大多是技术专家型，依靠个人的创作编著能力产出。

3.高热度作者分析

评论数通常代表着销量，能够直观地显示出哪些作者更加获得读者的追捧，销量也往往代表着市场的需求，出版社可以根据结果挖掘潜在的合作对象，销量排名靠前的作者通常在行业内具有较高的知名度和影响力，其作品更受关注，也更容易获得资源和机会。这样的作者具有更高的热度和更大的受众规模。这里利用平均每本图书的评论数分析了 Python 图书单本热度较高的作者，具体内容详见图 4-8：

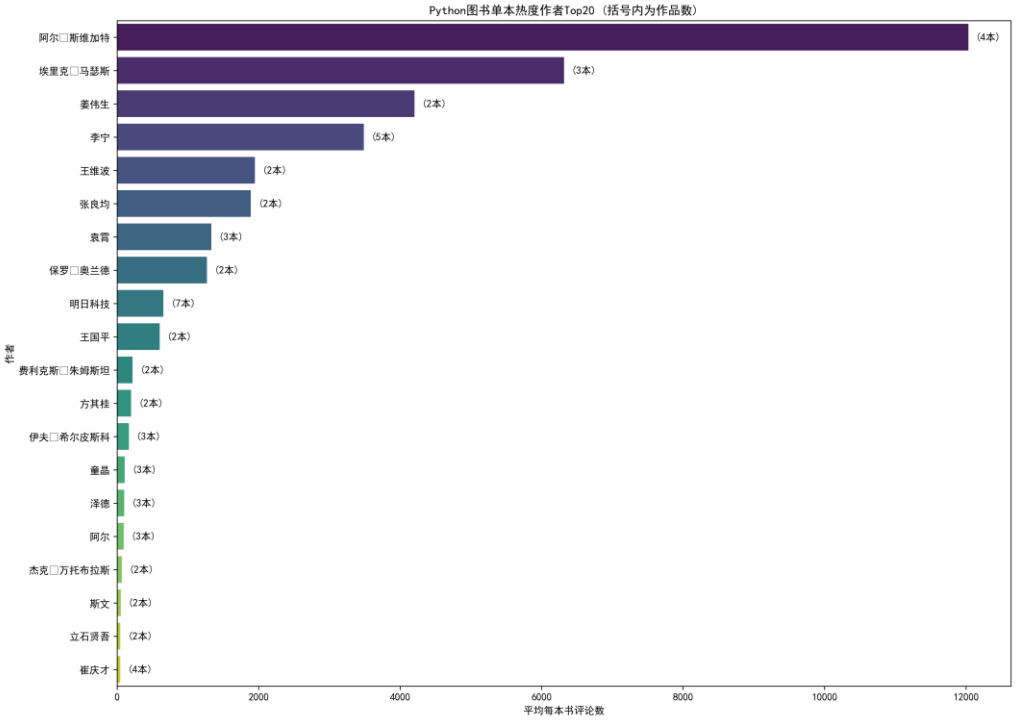


图 4-8：Python 图书单本热度作者 Top20

见图 4-8，阿尔斯维加特产出的图书销量遥遥领先，受到了读者的喜爱与追捧，埃里克马瑟斯的图书销量也已经过万，姜伟生的图书销量近万。国际作者在 Python 类图书的销售有着相对的统治力。高热度表明作者的作品符合市场需求，得到了读者的广泛认可，具有较强的市场竞争力。出版社对于热度较高的作者，可以加大出版投入，积极进行推广营销。

4.高质量作者分析

平均星级是读者对作者作品的直观评价体现，能反映出作者在整体创作水平上的表现。较高的平均星级通常意味着作者的作品在情节构思、文字表达、主题深度等方面得到了读者的广泛认可，具有较高的质量水准；反之，较低的平均星级则可能提示作者在某些方面存在不足，需要改进。具体内容详见图 4-9：

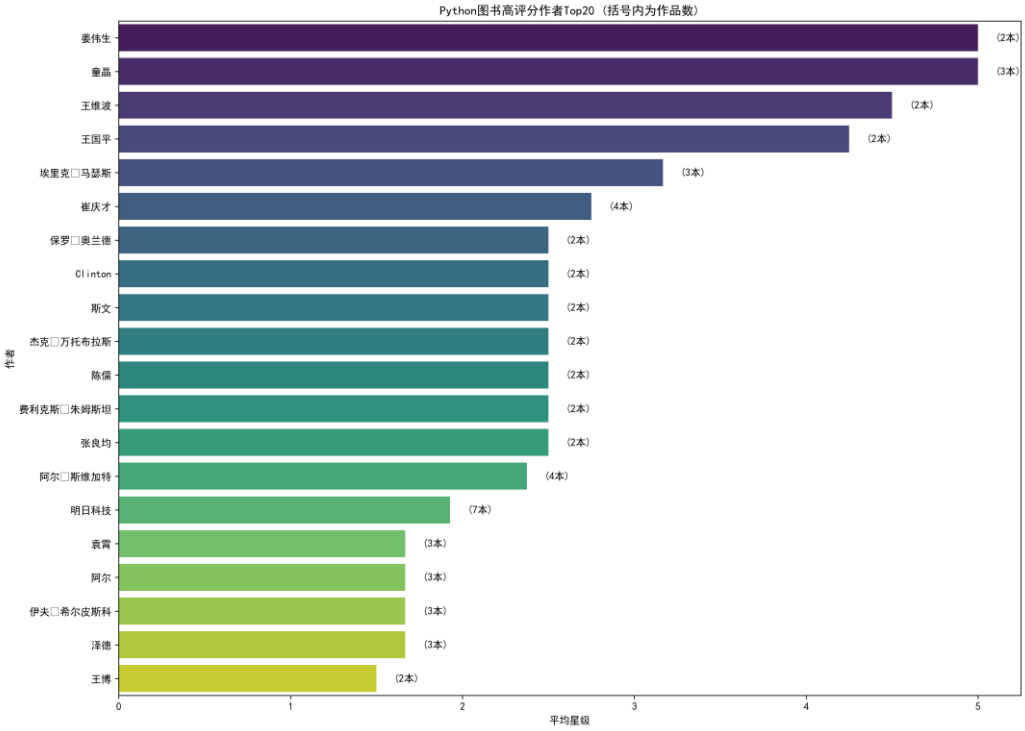


图 4-9：Python 图书高评分作者 Top20

见图 4-9，姜伟生、童晶等作者的平均星级较高，平均星级分析为读者提供了一个简洁有效的筛选参考。读者可以优先选择高平均星级的高产作者作品，降低挑选到低质量作品的风险，节省阅读时间和精力，更高效地找到符合自己兴趣和阅读期望的书籍。出版机构可以通过平均星级分析来评估高产作者的市场价值和潜力。对于平均星级高的作者，出版机构可以加大合作力度，给予更多的资源支持和推广投入，以获取更大的市场收益。

5.部分高产作者的代表作品及其销量、星级、价格分析

星级往往代表着读者对图书的满意度或者图书的质量，而销量直接反映了作者的市场占有程度，价格则反映出该作者图书的定价。head(5) 取作品数量最多的前 5 名作者，仅展示最具代表性的高产作者，可通过修改 head(n) 调整展示数量。从之前生成的字典中获取该作者的所有作品数据（包含书名、评论数等）并格式化写入作品详情。

具体代码详见代码清单 4-7：

代码清单 4-7

```
f.write("4. 高产作者的代表作品\n")
f.write("-" * 30 + "\n")
for author in author_stats.head(5)['author']:
    f.write(f"\n 作者: {author}\n")
```

```
        df = top_author_books[author].sort_values('comments_count', ascending = False)
        for i, (_, row) in enumerate(df.iterrows(), 1):
            f.write(f"  {i}. {row['short_title']}，评论数: {row['comments_count']:.0f}，星级: {row['stars']}，价格: {row['price']}元\n")
```

具体内容详见图 4-10：

```
作者: 阿尔·斯维加特
  1.  Python编程快速上手 让繁琐工作自..., 评论数: 48107, 星级: 4.5, 价格: 44.5元
  2.  Python编程轻松进阶 官方正版 出..., 评论数: 28, 星级: 5.0, 价格: 77.8元
  3.  Python编程三剑客：Python编..., 评论数: 3, 星级: 0.0, 价格: 192.7元
  4.  Python编程快速上手——让繁琐工作..., 评论数: 1, 星级: 0.0, 价格: 36.6元

作者: 埃里克·马瑟斯
  1.  Python编程从入门到实践 第3版 ..., 评论数: 17734, 星级: 4.5, 价格: 69.8元
  2.  Python编程三剑客第3版：Pyth..., 评论数: 974, 星级: 5.0, 价格: 134.3元
  3.  Python编程从入门到实战第3版 计..., 评论数: 242, 星级: 0.0, 价格: 69.8元

作者: 姜伟生
  1.  矩阵力量：线性代数全彩图解+微课+Py..., 评论数: 4486, 星级: 5.0, 价格: 138.0元
  2.  数学要素 全彩图解 + 微课 +Pyt..., 评论数: 3919, 星级: 5.0, 价格: 119.0元
```

图 4-10：部分高产作者代表作品分析图

不同作者有其不同的图书风格和类型，通过了解作者作品的星级可以间接了解到作者的口碑，出版社可以根据数据选择更适合自身发展的作者进行合作，也可以根据数据来优化自己的定价策略，或者对比以上头部作者的作品来优化图书的内容、方向、结构等等方面来提升销量。

4.2.2 图书的应用领域分析

通过对图书的应用领域的分析，出版社能够了解市场需求，发现热门领域和潜在的空白领域，从而有针对性地策划选题，提高图书的市场竞争力。也可以针对不同领域的读者群体制定精准的营销策略，选择合适的渠道进行推广，提高营销效果。

1. 关键词定义

为便于后续图书分类，本文通过定义大多数常见 Python 应用领域及其各自对应相关关键词，定义了一本 Python 类图书字典。其具体代码详见代码清单 4-8：

代码清单 4-8

```
    domains = {
        '数据分析': ['数据分析', '数据科学', 'pandas', '数据处理', '可视化', '图表', '统计'],
```

```
        '机器学习': ['机器学习', '人工智能', 'AI', '深度学习', '神经网络', '算法', '模型
'],
        '网络爬虫': ['爬虫', '网络爬虫', '抓取', '采集', '数据挖掘'],
         '自动化': ['自动化', '自动办公', 'office', 'excel', '效率', '工作流'],
         '金融量化': ['金融', '量化', '量化交易', '量化投资', '股票', '交易'],
         '图形界面': ['GUI', '图形界面', 'PyQt', 'Qt', 'UI', '界面开发'],
         '编程基础': ['入门', '基础', '从入门到精通', '精通', '实践', '零基础', '核心编
程'],
         '网站开发': ['Web', '网站', 'Django', 'Flask', '服务器', '后端'],
         '游戏开发': ['游戏', '设计', '游戏开发'],
         '数据库': ['数据库', 'SQL', 'NoSQL', 'MySQL', 'MongoDB'],
         '科学计算': ['科学计算', '数值计算', '数学', 'NumPy', '数值', '计算'],
         '青少年编程': ['青少年', '少儿', '儿童', '孩子', '趣味', '编程教育']
    }
```

2.领域识别函数

通过接收书名作为输入，返回该图书所属的应用领域。遍历预设领域与关键词，进行关键词匹配，通过检查图书书名中是否包含各个领域所对应的关键词来进行分类，如果没有匹配到任何领域，就标记为“其他”。

具体代码详见代码清单 4-9：

代码清单 4-9

```
def identify_domains(title):
    identified_domains = []
    title = title.lower()
    for domain, keywords in domains.items():
        for keyword in keywords:
            if keyword.lower() in title:
                identified_domains.append(domain)
                break
return identified_domains if identified_domains else ['其他']
```

3.领域统计

统计了每个领域所包含的图书数量，并按照数量降序排列来存储，方便后续的分析与可视化实现。首先创建空字典，用于存储每个领域及其出现次数，随后遍历数据集中的领域列表，df_books['domains'] 假设是数据框中的一列，存储每个样本的领域列表，逐行遍历该列， domains_list 为每个样本对应的领域列表（如某本书

可能属于多个领域）。最后统计每个领域的频次，最终 domain_counts 会累计所有样本中各领域的出现次数。

具体代码详见代码清单 4-10：

代码清单 4-10

```
domain_counts = {}
for domains_list in df_books['domains']:
    for domain in domains_list:
        if domain in domain_counts:
            domain_counts[domain] += 1
        else:
            domain_counts[domain] = 1
domain_df = pd.DataFrame({'domain': list(domain_counts.keys()),
'count': list(domain_counts.values())}).sort_values('count', ascending=False)
```

4.绘制应用领域分布图

能够以可视化的方式清楚地展示不同应用领域图书的数量或占比情况，通过可视化展示各个领域 Python 类图书的数量可以快速地识别出哪些应用领域最热门，哪些应用领域相对冷门一些，也可以据此揭示供需关系。对于出版社来说，分布图能为选题策划提供重要参考。出版社可以根据各领域图书的分布情况，结合市场需求和自身资源优势，合理规划选题方向，加大对热门领域图书的出版力度，同时也可适当关注一些有潜力的新兴领域，提前布局相关图书的出版，从而为不同的角色提供决策上的支持和依据。具体内容详见图 4-11：

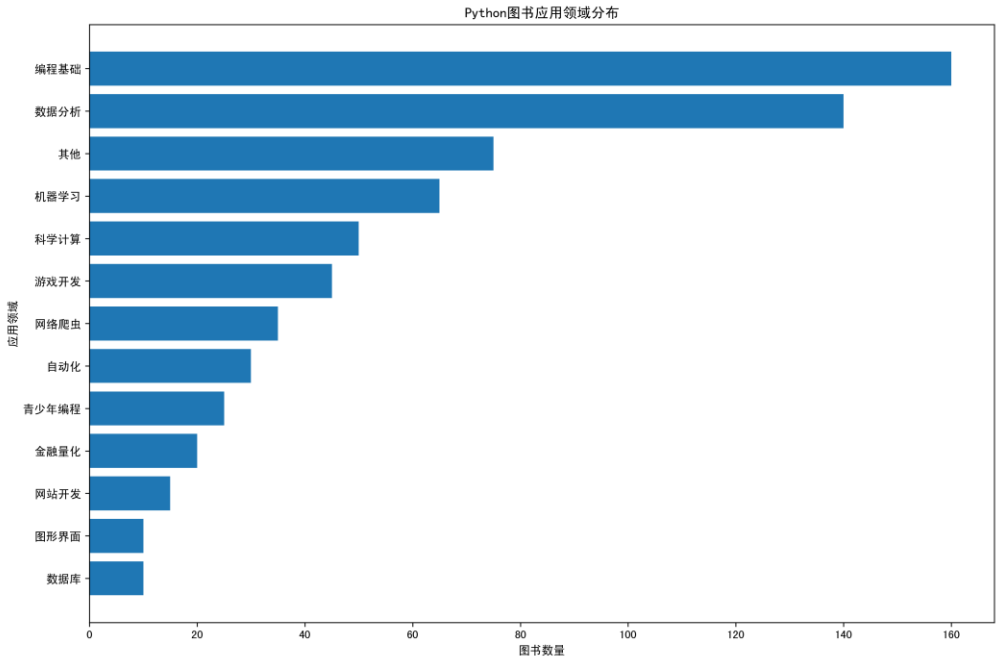


图 4-11：Python 图书应用领域分布图

见图 4-11，在编程基础这一领域的图书是最多的，这代表着在这一领域内市场需求量最大，编程基础、数据分析等热门领域的图书技术代表着当前 Python 类图书的主流方向，但是出版社应当注意这种热门领域的市场可能已经达到了饱和，新书需要差异化定位，冷门的领域反而更可能是没有被充分开发的市场。

同时对于读者来说，可以通过此分析结果了解到编程基础、数据分析、机器学习等领域资源比较丰富，而图形界面、数据库等领域的资源比较缺乏，帮助读者进行更高效的了解与选择需要的图书。

5.各领域图书的热度分析

图书的总评论数往往代表着该图书的热度，通过量化不同领域的图书的热度差异，可以发现市场供需的缺口或者市场机会，也可以用来评估这个领域的成熟度和竞争形势。具体内容详见图 4-12：

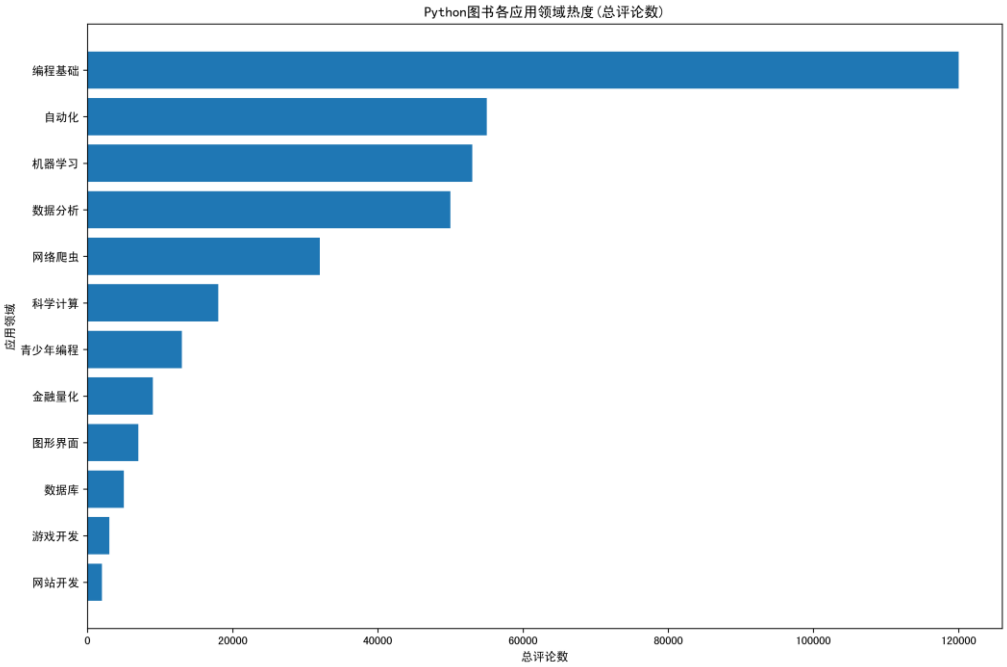


图 4-12：Python 图书各应用领域热度图

见图 4-12，编程基础的热度是最高的，其次是自动化、机器学习、数据分析等等领域，对于开发者来说，可以优先去学习这些热度较高的技术，提升自己的就业竞争力，企业也可以据此调整投入。

对于出版社来说。可以针对性地调整对各个领域的资源投入，优先将资源投入编程基础、自动化这类热度比较高的领域，加大对热门领域图书的选题开发力度。需要注意的是，热度较高的领域一般发展比较成熟，出版社需要追求差异化地去出版图书。

读者可以通过了解图书领域热度，发现当下热门的知识领域和研究方向并根据自己的兴趣去选择。

6.应用领域满意度分析

图书的星级可以有效地反映出读者的满意度，是读者对于图书的综合性评分，能够衡量出该领域内图书的平均质量和平均评价。具体内容详见图 4-13：

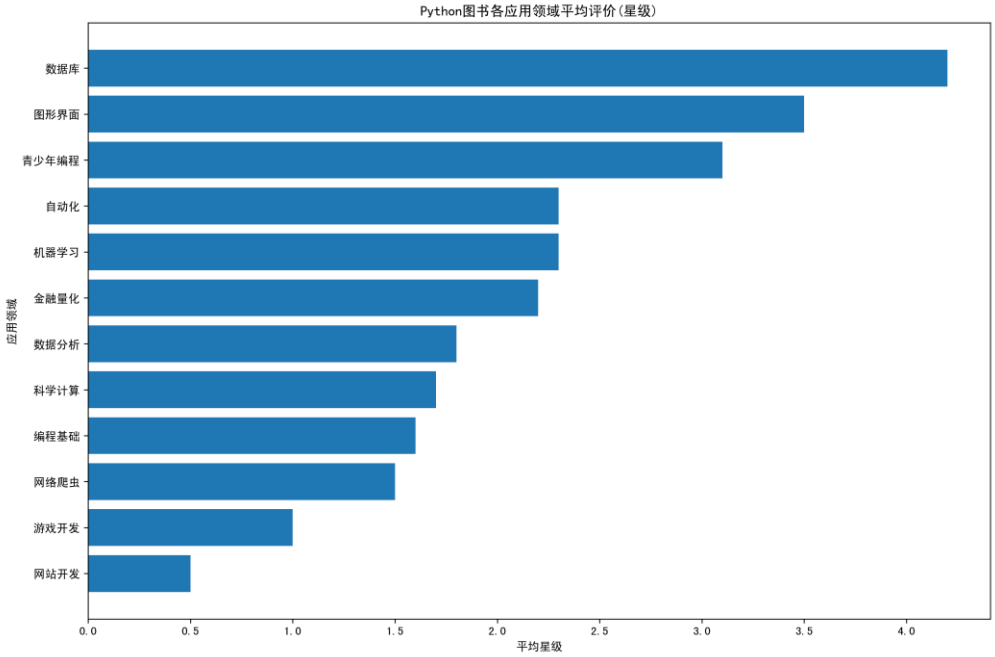


图 4-13：Python 图书各应用领域平均评价图

经分析图 4-13 可知，数据库这一应用领域的图书的读者平均评价是最高的，这种较高评价的领域的图书一般内容丰富易懂、质量较好，更容易获得新读者的信任。而较低星级的领域往往存在着技术过时、内容晦涩难懂或者因为营销过度造成预期落差等等的情况。

对于开发者来说，可以优先学习高评价的领域，这样试错的成本比较低。对于出版社来说，应当吸取高星级领域的图书的优点，对低星级领域的图书进行改善。

对于读者来说，可以优先选择高星级领域的图书，可以结合详细评论仔细甄选。

7.各应用领域最具代表性的图书分析

在图书市场中，评论数作为重要的数据指标，能够近似映射图书的实际销量情况，评论数作为客观数据，也能够直观反映读者对图书的参与度。基于评论数筛选各应用领域的代表图书，能有效地提升筛选结果的可靠性。具体内容详见图 4-14：

```
5. 各应用领域代表图书(评论数最多)
------------------------------
编程基础:  Python编程快速上手 让繁琐工作自动化 第2版 Python编程从入门到实..., 评论数: 48107, 星级: 4.5
自动化:  Python编程快速上手 让繁琐工作自动化 第2版 Python编程从入门到实..., 评论数: 48107, 星级: 4.5
机器学习:  Python编程从入门到实践 第3版 Python编程入门经典，自学利器，数据..., 评论数: 17734, 星级: 4.5
数据分析:  Python编程从入门到实践 第3版 Python编程入门经典，自学利器，数据..., 评论数: 17734, 星级: 4.5
网络爬虫:  Python编程从入门到实践 第3版 Python编程入门经典，自学利器，数据..., 评论数: 17734, 星级: 4.5
科学计算:  矩阵力量：线性代数全彩图解+微课+Python编程 鸢尾花书 从加减乘除到机器..., 评论数: 4486, 星级: 5.0
青少年编程:  小学生Python创意编程（视频教学版） 本书按照入门开发者的思维方式编写，非..., 评论数: 5461, 星级: 5.0
金融量化:  Python股票量化交易从入门到实践 股票炒股趋势技术分析书籍，量化投资金融科..., 评论数: 3989, 星级: 5.0
图形界面:  Python Qt GUI与数据可视化编程 PyQt5教程书籍 pyqt5快速..., 评论数: 3538, 星级: 4.5
数据库:  Python编程实战100例 100个编程实例 1390分钟视频讲解 核心编程..., 评论数: 2379, 星级: 5.0
```

图 4-14：各应用领域代表图书分析图

见图 4-14，以上这些图书不仅在内容质量上获得认可，还在市场传播中形成了广泛影响力。

出版社可以通过了解这些热门图书的内容架构、选题方向、包装设计、营销策略等要素，精准地把握市场需求和读者偏好，进而优化自身的图书内容。

面对市场上海量的图书信息，读者往往难以快速辨别优劣，而评论数高的图书相当于经过了市场和读者群体的筛选，能够帮助读者更高效地选择图书。

4.2.3 图书销量的相关性分析

图书的评论数量往往代表着图书的销量或者热销程度，当当网能够爬取的销售字段中并无销量，当缺乏直接销售数据时，评论数是最可靠的销量代理数据，我们可以利用评论总数近似地看作销量来进行分析。

通过分析图书销量与不同特征值之间的相关性，如与价格、作者、星级等的关系，出版社可以明确哪种图书更受市场欢迎，就可以集中资源在这些部分，提高资源利用效率，降低成本，同时也有助于进行精准营销。

1.特征工程

在原有的选取了原有的特征值的基础上，创建了新的特征能够更好地揭示数据中存在的关系，“price_per_comment”可以用来代表“用户评论的成本”，可以揭示价格与评论数之间的关系，“+1”可以防止除零错误。“log_comments”对评论数进行对数变换可以使其更接近正态分布，提升分析的准确性，“log_price”同理。具体代码详见代码清单 4-11：

代码清单 4-11

```
df_analysis['price_per_comment'] = df_analysis['price'] /
(df_analysis['comments_count'] + 1)
df_analysis['log_comments'] = np.log1p(df_analysis['comments_count'])
df_analysis['log_price'] = np.log1p(df_analysis['price'])
df_analysis['title_length'] = df_analysis['title'].str.len()
df_analysis['author_count'] = df_analysis['author'].str.count(',') + 1
```

2.相关性计算与可视化实现

分析销量与各个因素之间的相关性可以助力出版机构决策，有助于发掘更加高效的市场策略，进而优化资源分配、实现精准营销。

从数据框 df_analysis 中筛选出类型为数值的列（包括整数、浮点数等），排除文本、日期等非数值型特征。随后计算所选数值特征之间的相关性，并绘制完整的相关性热力图。

具体代码详见代码清单 4-12：

代码清单 4-12

```
# 计算所有数值特征之间的相关性
numeric_features = df_analysis.select_dtypes(include=['number']).columns.tolist()
correlation_matrix_all = df_analysis[numeric_features].corr()

# 绘制完整的相关性热图
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix_all, annot=True, cmap='coolwarm', vmin=-1, vmax=1, fmt='.2f')
plt.title('所有数值特征的相关性矩阵')
plt.tight_layout()
plt.savefig(f'{result_dir}/所有特征相关性矩阵.png', dpi=300)
plt.close()
```

绘制销量与其他因素的相关性矩阵能够直观地呈现哪些因素与销量有显著相关性，如评论数、星级评分等，帮助明确影响图书销量的关键要素。可以判断关系方向，确定因素与销量是正相关还是负相关，发现一些不易察觉的变量关系，为出版社制定出版策略提供思路，可据此调整出版计划。

通过图示可以快速地识别出变量之间的关联，直观地展示出哪些变量之间正相关、负相关或者无关。具体内容详见图 4-15：

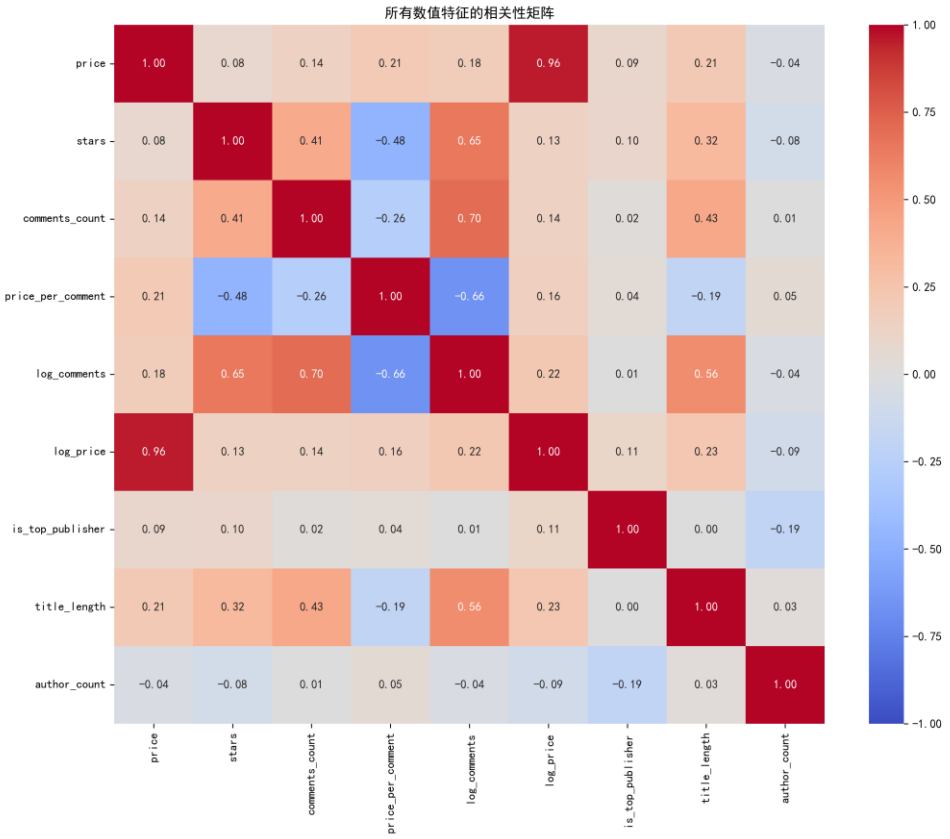


图 4-15：相关性矩阵图

在相关性矩阵图中，各单元格数值表征变量间的线性相关系数。为以红色系编码正相关关系，颜色饱和度与正相关程度呈正相关；蓝色系则对应负相关关系，其明度变化与负相关强度形成映射，通过色彩梯度直观呈现变量间线性关联的方向与程度。每个色块上的数字大小也直接映射了相关性的大小。

见图 4-15，销量与价格的相关性为 0.14，呈现弱正相关，这可能是因为部分用户认为高价代表着高质量，高价商品的销量未必就低，用户更愿意为了优质商品而买单，也有可能高价商品多来自于知名出版社或者知名作者，用户更加新人，其中存在着品牌效应的影响。销量与价格对数变换后的相关性分析同理。

销量与星级的相关性为 0.41，呈现中等正相关，这代表图书的星级越高，则销量就会更高，不过这种影响有限，用户在购买图书时会参考星级，高星级的图书往往代表着图书的质量好，所以用户的购买率和复购率就会增多，这样就形成了正向的循环，高星级图书的销量稳定性会高于低星级的图书。

销量与用户评论成本的相关性为-0.26，呈现弱负相关，说明评论成本仅对销量有轻微影响，用户更关注产品本身，高成本评论可能来自于高价商品，用户购买后会因为付出了更高昂的价格而更愿意评论，低成本评论可能来自于低价商品，用户因购买门槛低而随意评论。

销量与销量的对数变换相关性为 0.70，呈现高度正相关，销量的对数变换与原始销量数据高度线性相关，说明对数变换保留了原始数据的趋势，对数变换后仍能够反映出原始数据的分布规律，改善了统计特性，提升分析的准确性。

销量与出版社的相关性为 0.02，销量与作者的相关性为 0.01，呈现非常弱的正相关。出版社因素和作者因素（如知名度、历史作品表现等）与销量几乎无显著关联。

销量与图书书名长度的相关性为 0.43，长书名通常包含更具体的关键词、场景或承诺，短书名则更加精简。出版社可能对预期畅销的图书刻意设计标题，集中突出卖点。

3.星级、价格与销量之间的关系分析

可直观地观察到星级、价格和销量之间的相互关系，能够识别出价格敏感区间，同时结合星级，分析高星级图书是否在某些价格段更具销量优势，为出版社的出版策略和定价方式提供依据。读者能通过散点图了解不同星级和价格图书的市场表现，根据自己的需求和预算，选择性价比高的图书。

具体内容详见图 4-16：

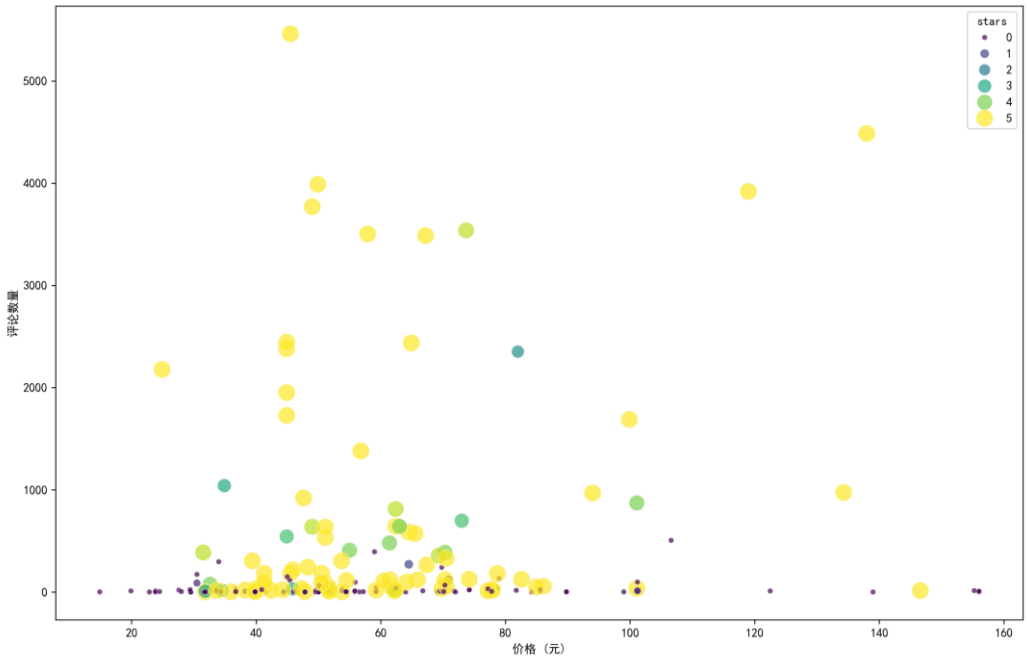


图 4-16：Python 图书星级与销量的关系图

见图 4-16，右上角标识用颜色代表星级，横轴表示价格，纵轴表示评论数量。

价格在 40 到 60 的区间内出现销量高峰，并且高星级也主要集中在百元以下，百元以上高星级且高销量是比较少的。价格在 40 到 60 区间内实现了销量与星级双高，说明此区间是市场的核心区，而百元以上的商品难以同时维持高销量和高星级，出版社可以适当地调整自己的定价策略。

4.与销量相关性最高的特征值分析

可以精准地定位关键因素，明确对销量影响最大的特征值，通过分析特征值与销量的相关性以及散点图的分布规律，可以量化关键特征值对销量的影响程度，为决策提供数据支撑。出版社可以据此合理分配资源，对于对销量影响大的因素，可投入更多资源，以提升销量。

具体内容详见图 4-17：

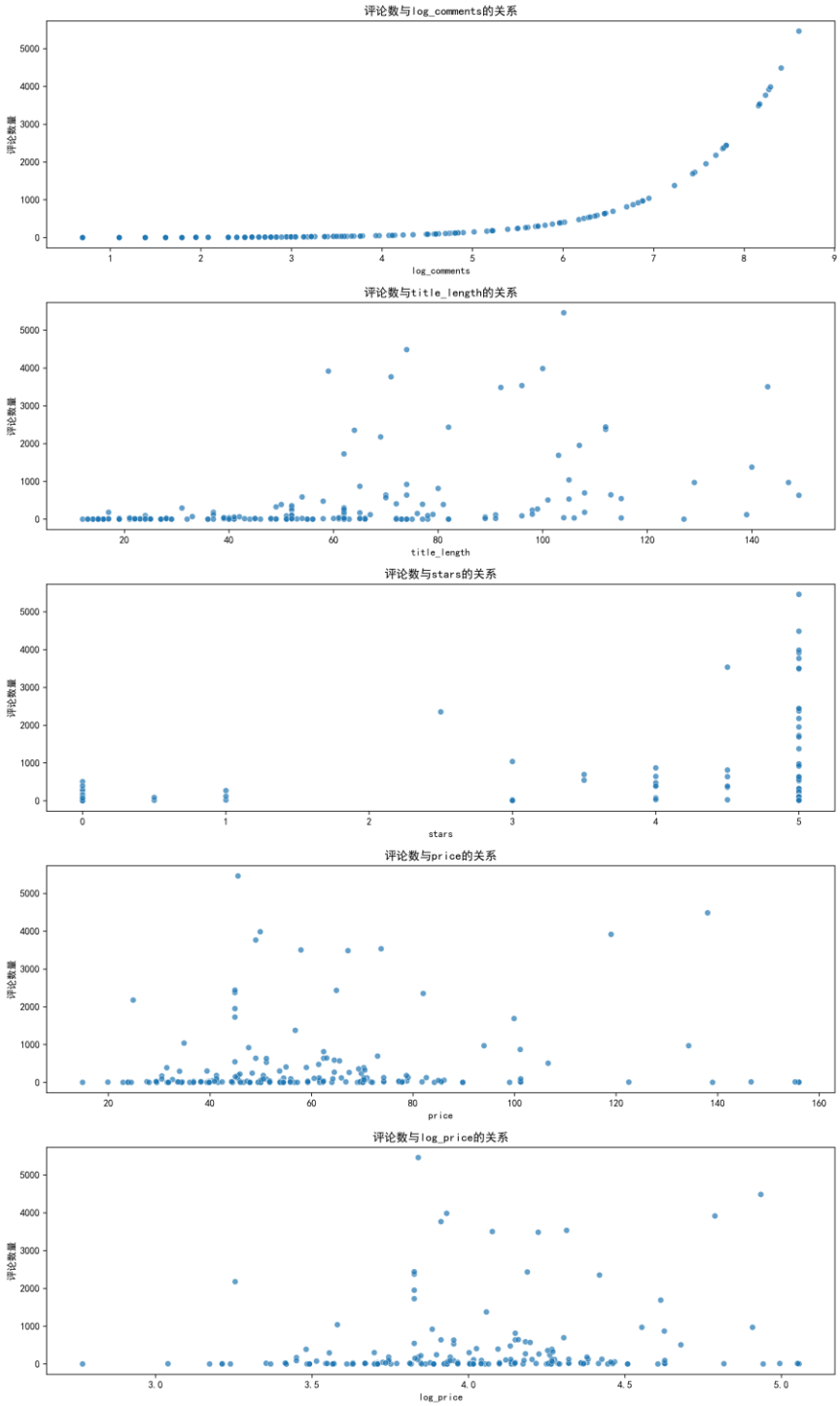


图 4-17：五类与销量相关性最高的特征值散点图集

见图 4-17，横轴代表特征值，纵轴代表评论数量（即销量），可以通过观察点的密集程度和位置得知销量与各特征值之间的关系。

评论数与评论数取对数散点图原始评论数与经过对数变换后的评论数进行可视化展示，散点呈现出类似曲线的趋势，说明原始评论数在一定范围内遵循特定的数学规律。

评论数量与书名长度的散点图相对比较分散，但可以观察出评论数量较高的图书分布在书名长度 60 到 120mm 的长度范围内。

评论数量与星级的散点图中观察到图书星级越高，其评论数量往往越高，二者呈现出正相关的关系。

评论数量与价格中，散点多聚集于价格在 20 至 100 的范围，在这个区间内，评论数量较多的图书也比较多，只有少部分评论较多的图书分布在价格 100 以上，这类图书即使价格和销量都比较高，说明图书的质量受到读者的喜爱。

评论数量与价格取对数的散点图中散点从左下到右上分布，说明二者呈现正相关趋势，验证了价格越高，销量越多的正相关关系。

评论数对数、书名长度、星级、价格及其对数这五个特征值对销量的影响最大，通常评论数越多，说明图书越受关注，较多的评论数能增加图书的曝光度和口碑传播，吸引更多的潜在读者购买。适中长度的书名或许更有利于吸引读者，从而对销量产生积极影响。星级代表着图书质量得到读者的满意度，通常星级越高，越能获得读者的信任。价格对销量的影响较为复杂，如果价格过高，可能会抑制销量，如果价格过低可能会让读者对图书质量产生怀疑，也不利于销量，出版社应当选择更适中的定价策略。而对价格取对数后，可能更能体现价格在不同量级上对销量的影响。

通过分析这五个特征值与销量的关系，能够帮助出版社更深入地了解图书市场，制定更有针对性的营销策略，如根据图书的星级和评论数进行精准推广，根据价格与销量的关系优化定价策略，以及考虑书名长度对图书吸引力的影响等，以提高图书的销量和市场竞争力。同时也为更深入的研究提供了支持。

4.2.4 图书销量的预测分析

通过对销量进行预测，出版社可以合理安排图书的库存，防止因印量过多导致库存积压，减少仓储成本和资金占用，也能确保市场有足够的图书供应，避免出现缺货的情况，有助于合理调配生产设备和人力，提高生产效率，降低生产成本。

根据预计销量，出版社也可以更科学地分配营销资源，例如确定广告投放规模、促销活动力度以及选择合适的营销渠道，使营销投入更精准有效。对于预计销量较

大的图书加大营销资源的投入，进行广泛地宣传或者多进行推广活动。对于预计销量较小的图书，可以选择有针对性的推广，提高自身的市场竞争力。

销量预测数据同时也可以帮助出版社进行财务预估，可以帮助出版社在成本控制上有的放矢。如果预计的销量比较低，则可以在保证质量的基础上适当地压缩生产成本，如果预计的销量比较高则可以适当地增加生产以及营销的投入。

图书的评论数可以近似地看作是图书的销量，这部分的研究通过对比多个模型，最终选择最优模型为梯度提升，它是一种比较强大的机器学习算法，适合处理中小规模的数据集和非线性关系，能够有效地处理高维特征空间。

1. 特征工程

首先进行数据准备，根据图书销量的相关性分析结果，选取了评论数对数、书名长度、星级、价格及其对数这五个与销量相关性最大的特征值。

具体代码详见代码清单 4-13：

代码清单 4-13

```
# 标题长度
df_books['title_length'] = df_books['title'].apply(len)
# 对数变换缓解偏态
df_books['log_comments'] = np.log1p(df_books['comments_count'])
df_books['log_price'] = np.log1p(df_books['price'])
```

2.进行数据划分

按照 70%和 30%的比例划分训练集和测试集。

具体代码详见代码清单 4-14：

代码清单 4-14

```
# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state
= 42)
print(f"训练数据形状: X_train={X_train.shape}, y_train={y_train.shape}")
print(f"测试数据形状: X_test={X_test.shape}, y_test={y_test.shape}")
```

3.模型训练与评估

对多种模型进行评估，包括线性回归模型、随机森林模型、岭回归模型、梯度提升回归模型、Lasso 模型，线性回归模型一般用于存在线性关系的分析当中，随机森林模型则可以捕捉模型之间复杂的交互问题，梯度提升树常用于预测数值型目标变量，可以用于特征的选择和数据的探索，岭回归常用于处理多重共线性问题，可以提高模型的泛化能力，Lasso 模型能够进行特征选择，并且常用来处理高维的数据。具体代码详见代码清单 4-15：

代码清单 4-15

```
from sklearn.linear_model import LinearRegression
base_models = {
    '线性回归': LinearRegression()
}
from sklearn.linear_model import Ridge
base_models['Ridge 回归'] = Ridge(alpha=1.0)   # alpha 为正则化强度
from sklearn.linear_model import Lasso
base_models['Lasso 回归'] = Lasso(alpha=0.1)   # alpha 控制特征筛选严格程度
from sklearn.ensemble import RandomForestRegressor
advanced_models['随机森林'] = RandomForestRegressor(
    n_estimators=200,
    max_depth=20,
    min_samples_split=2,
    random_state=42
)
from sklearn.ensemble import GradientBoostingRegressor
advanced_models['梯度提升树'] = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=5,
    random_state=42
)
# 评估所有模型
model_results = {}
y_preds = {}
for model_name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    model_results[model_name] = {'r2': r2, 'rmse': rmse}
    y_preds[model_name] = y_pred
```

```
    # 打印模型性能
    for model_name, metrics in model_results.items():
        print(f"{model_name}: R²={metrics['r2']:.4f}, RMSE={metrics['rmse']:.2f}")
    # 找出最佳模型
    best_model_name = max(model_results.items(), key=lambda x: x[1]['r2'])[0]
    print(f"\n 最佳模型: {best_model_name}, R²:
{model_results[best_model_name]['r2']:.4f}")
```

基于 R^2 的值选择出表现最好的模型，R^2 的值是决定系数，也是衡量回归模型拟合程度的一个重要指标。其取值范围在 0 到 1 之间，R^2 越接近 1，说明模型对数据的拟合效果越好，较高的 R^2 值在一定程度上也表示模型具有较好的预测能力。具体内容详见图 4-18：

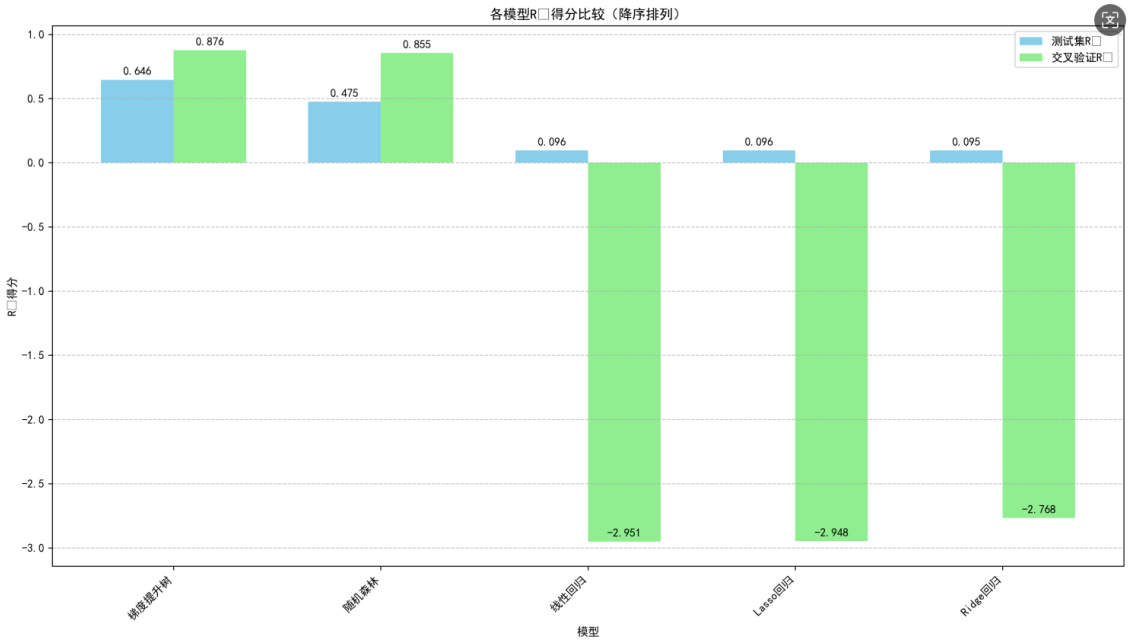


图 4-18：各模型 R^2 值柱状图

图 4-18 中，横轴为各个模型，纵轴为 R^2 值，可以看出梯度提升树模型的 R^2 是最大的，说明当前梯度提升模型的表现最好，这也表示图书的销量受复杂因素的影响，而非线性关系显著，所以这里采用梯度提升模型进行预测。

4.预测结果

使用之前选出的最佳模型对全部特征数据 X 进行预测，具体内容详见图 4-19：

actual_comments	predicted_comments
5	4.971699120801111
133	124.43954496043595
0	0.028970638049188088
969	979.8736711954907
33	34.46037207608782
0	-0.023057171
643	639.412497863038
43	43.58298167137192
0	-0.001903118
1	0.9175889098292074
42	43.49383439356728
921	1016.1601058479843
1	1.0100005145714863
20	19.12418409094042
0	0.005226706862471396

图 4-19：销量预测结果图

4.3 热销图书用户评论数据情感分析

本部分的研究针对评论数据进行了情感分析，用户会在评论内容中用文字描述具体的感受，评分也天然地反映了用户的满意程度，可以用于电商评论情感分析、用户反馈等等场景。

对用户评论进行情感分析能够帮助作者优化图书内容，通过分析读者评论中的情感倾向，了解读者对图书内容的喜好和不满，从而有针对性地对后续作品进行改进和优化。出版社可以根据情感分析结果，对图书质量或者服务等方面进行优化。分析图书用户评论数据中的情感倾向，也可以帮助市场研究人员了解当前图书市场的整体趋势，为市场的发展和优化提供理论研究。

分析过程中首先利用 K-means 聚类算法对图书进行了聚类，识别并选取了热销类图书并对其评论内容进行了情感分析，情感分析部分使用了 LDA 进行主题建模。利用 TF-IDF 将文本转换为数值特征，同时使用了 SVM 判断好评和差评，并提取出正面和负面评论的关键词。

4.3.1 基于 K-means 聚类的热销图书分析

在图书市场的数据分析中，对图书进行聚类处理是深度挖掘用户需求、精准把握市场趋势的重要步骤。通过使用 K-means 聚类算法，将海量图书数据划分成具有相似特征的不同群组。

选取最热销的图书聚类用于后续情感分析，热销图书往往引领市场潮流与阅读趋势，具有显著的实际意义与价值，热销图书的评论数据基数庞大，能够有效地降低误差，确保情感分析结果的可靠性与稳定性。

1. 选择特征

选择用于聚类的特征值为评论数、星级和价格，并进行标准化处理，因为 K-means 对特征的尺度比较敏感，进行标准化可以保证各特征值权重相等。

具体代码详见代码清单 4-16：

代码清单 4-16

```
print("正在进行热销书聚类分析...")
cluster_features = ['comments', 'stars', 'price']
cluster_df = books_df[cluster_features].dropna()
scaler = StandardScaler()
scaled_features = scaler.fit_transform(cluster_df)
```

2.肘部法则确定最佳聚类数

使用肘部算法不需要事先对数据有深入的了解或对聚类数有预设的判断，它完全基于数据本身的特征来确定聚类数，避免了主观因素的干扰。并且它适用于各种类型的数据，具有较高的通用性和灵活性，能够使得聚类结果更加合理和可靠。

此处尝试测试 K=1 到 10 的聚类，记录每个 k 值的惯性值，利用了肘部法则绘制惯性值变化曲线，继而帮助选择最佳 K 值。这样选择 K 值更具有客观性和科学性。具体代码详见代码清单 4-17：

代码清单 4-17

```
inertia = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(scaled_features)
    inertia.append(kmeans.inertia_)
```

具体内容详见图 4-20：

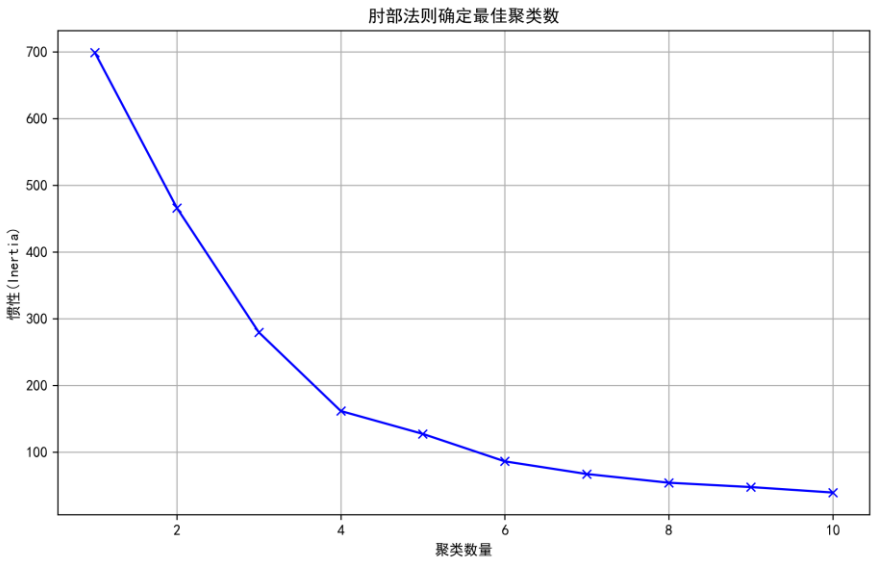


图 4-20：肘部算法确定最佳聚类数折线图

见图 4-20， K 值介于 1 至 3 时，惯性快速下降，说明了增加聚类数能够显著地提升聚类的效果，而在 K = 3 后，曲线趋于平缓，增加 K 值对惯性的改善效果有限，所以最佳 K 值应当选择拐点 K = 3。

3.聚类并识别热销图书

使用肘部算法确定的最佳聚类数进行聚类，将聚类结果进行分析，按照簇进行分组并计算各个特征值的均值，打印出每个聚类的平均评论数、星级和价格，并将评论数均值最高的聚类识别为热销图书。

具体代码详见代码清单 4-18：

代码清单 4-18

```
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
books_df.loc[cluster_df.index, 'cluster'] = kmeans.fit_predict(scaled_features)
cluster_analysis = books_df.groupby('cluster')[cluster_features].mean()
print("\n 聚类分析结果:")
print(cluster_analysis)
bestseller_cluster = cluster_analysis['comments'].idxmax()
bestsellers = books_df[books_df['cluster'] == bestseller_cluster]
```

具体内容详见图 4-21：

cluster	comments	stars	price
0	1041.28125	4.75	61.648854166666666
1	27.985185185185	0.0296296296296	60.48348148148148
2	82789	4.75	33.05

图 4-21：聚类结果展示图

具体内容详见图 4-22：

A	B	C	D	E	F	G	H	I	J	K
	title	author	publisher	img	price	comments	stars	comments_detail	publisher_categ	cluster
14	Python编程快速	阿尔·斯维加特，	人民邮电出版社	http://img3m4.d	44.5	48107	4.5	[]	人民邮电出版社	2
358	Python编程三剑	未知		http://img3m4.d	21.6	117471	5	[]	其他出版社	2

图 4-22：聚类书籍列表展示图

基于图 4-22 可识别出 cluster2 为热销图书，并利用此分类数据进行后续分析。

4.3.2 基于 LDA 的评论主题分布分析

使用了 LDA 算法进行主题建模，LDA 能够自动发现文本数据中潜在的主题。在评论数据中存在着多种不同的主题，如产品的不同功能、服务的不同方面等，LDA 可以将这些隐藏的主题挖掘出来。

出版社能够更有针对性地了解用户对产品各个方面的评价。持续性地监测评论主题分布的动态变化，能捕捉市场趋势与用户需求的演变，将自家图书评论与竞品图书评论进行主题分布对比，可清晰了解自身在市场中的优势与劣势，并对自家图书进行优化。

1.文本向量化

使用 TF-IDF 对文本进行向量化工作，将文本特征转换为机器可以识别的数值特征。限制只保留词频最高的 1000 个词，过滤低频词和噪声，有效避免数据稀疏性问题。

具体代码详见代码清单 4-19：

代码清单 4-19

```
print("正在进行 TF-IDF 转换...")
tfidf_vectorizer = TfidfVectorizer(max_features=1000)
tfidf_matrix = tfidf_vectorizer.fit_transform(processed_comments)
feature_names = tfidf_vectorizer.get_feature_names_out()
```

2.LDA 主题数量优化

合适的主题数量往往能使模型更精准地捕捉文本数据中的语义结构和潜在主题。若主题数量过多，会导致模型过拟合，使结果缺乏代表性，若主题数量过少，模型则会欠拟合，无法充分挖掘数据中的复杂信息。合理的主题数量可以避免主题过于细化或粗略，有助于快速把握文本中的主要内容和核心思想。

定义测试主题数从 3 到 14（可以自由修改，小规模数据的主题数下限一般大于 2，上限一般小于 20）。随后初始化三个评价指标，一致性得分用来衡量语义的合理性，数值越高越好，高一致性表示主题内的词语义相近。困惑度用来衡量模型对未见文档的预测能力，值越低越好，低困惑度表示主题分布更加紧凑。对数似然值用来衡量模型对训练数据的拟合程度，值越高表示模型越好，需要结合一致性来避免过拟合。通过遍历不同都主题数，训练 LDA 模型。

具体代码详见代码清单 4-20：

代码清单 4-20

```
    print("正在优化 LDA 主题数量...")
    topic_range = range(3, 15)
    coherence_scores = []
    perplexity_scores = []
    log_likelihood_scores = []
    for n_topics in topic_range:
        print(f"测试主题数量: {n_topics}")
    # 计算评估指标
        perplexity = lda_model.perplexity(tfidf_matrix)  # 困惑度（越低越好）
        log_likelihood = lda_model.score(tfidf_matrix)  # 对数似然度（越高越
好）
    # 自定义主题一致性分数（通过词对共现估计）
        topic_words = [feature_names[i] for i in topic.argsort()[:-11:-1]]
        coherence = sum(1 for pair in word_pairs if ' '.join(pair) in ' '.join(processed
_comments))
    coherence = 0
        for topic in topic_words:
            word_pairs = [(topic[i], topic[j]) for i in range(len(topic)) for j in
range(i+1, len(topic))]
            topic_coherence = sum(1 for pair in word_pairs if ' '.join(pair) in '
'.join(processed_comments))
            coherence += topic_coherence / (len(word_pairs) if word_pairs else 1)
# 避免除零错误
            coherence_scores.append(coherence / len(topic_words))
```

3.确定最佳主题数

综合考虑多项指标，能够找到使模型既不过度拟合训练数据，又能充分挖掘数据中潜在结构的主题数，从而提高模型的准确性。单一指标可能存在局限性，无法全面反映模型的性能。多项指标综合考虑可以从不同角度评估模型，减少因单一指标的片面性导致的错误判断，使确定的最佳主题数更具稳定性和可靠性。

通过加权平均：困惑度 40%+一致性 40%+对数似然度 20%的计算方法，综合选择最佳主题数，确保模型既拟合数据又具有语义可解释性。

具体代码详见代码清单 4-21：

代码清单 4-21

```
# 一致性和对数似然度越高越好
norm_coherence = [(score - min(coherence_scores)) / (max(coherence_scores) - min(coherence_scores))
        for score in coherence_scores]
norm_likelihood = [(score - min(log_likelihood_scores)) / (max(log_likelihood_scores) - min(log_likelihood_scores))
        for score in log_likelihood_scores]
# 综合得分 - 赋予不同权重
combined_scores = [0.4*p + 0.4*c + 0.2*l
        for p, c, l in zip(norm_perplexity, norm_coherence, norm_likelihood)]
best_idx = combined_scores.index(max(combined_scores))
optimal_topics = topic_range[best_idx]
```

4.训练模型并提取关键词

训练模型并提取关键词能够从大量文本中挑选出最具代表性的信息，提高模型训练效率。通过提取关键词，可以快速了解文本的大致内容和重点，有助于对大量文本进行快速浏览和筛选。

使用最佳主题数来训练模型，每个主题的关键词通过 components_ 属性提取，按权重排序（ argsort() 返回升序索引， [:-21:-1] 表示取 20 个权重最高的词）

具体代码详见代码清单 4-22：

代码清单 4-22

```
lda = LatentDirichletAllocation(n_components=optimal_topics, random_state=42)
lda.fit(tfidf_matrix)
topic_report = []
for topic_idx, topic in enumerate(lda.components_):
    top_words_idx = topic.argsort()[:-21:-1]  # 提取前 20 个关键词
    top_words = [feature_names[i] for i in top_words_idx]
    topic_report.append(f"主题 #{topic_idx+1}: {', '.join(top_words)}")
```

5.指标评估对比

绘制三个指标（困惑度、一致性、对数似然度）随主题数量的变化曲线，通过观察三个指标的变化趋势，可以更直观地找到使模型性能最佳的主题数量。以图形方式展示指标变化，比单纯的数值更直观易懂。

具体内容详见图 4-23：

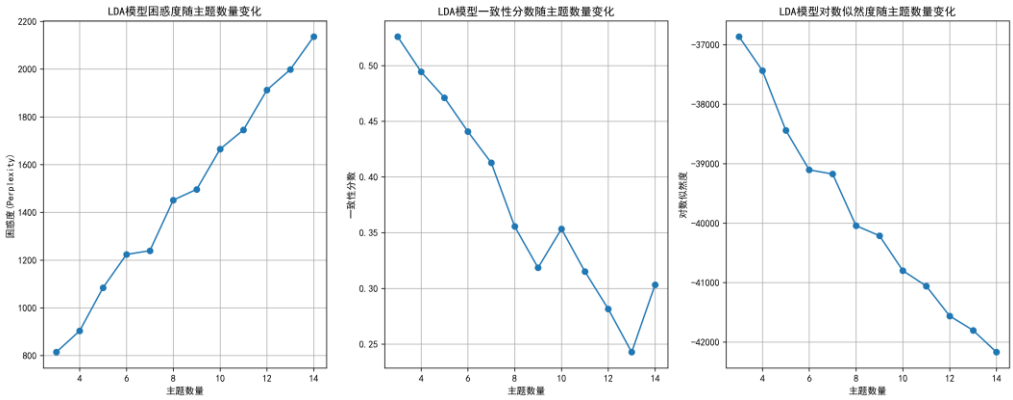


图 4-23：各指标分数随主题数量变化图

见图 4-23，LDA 主题数量为 3 时，其困惑度最低，一致性和对数似然度最高。说明主题数量为 3 时，模型对数据的拟合程度好，在预测未知数据时表现更优，能准确地对新文本进行主题分类和生成，泛化能力较强。且模型提取的主题内的关键词之间紧密相关。所以选择 LDA 主题数量为 3。

6.主题与关键词汇总

将主题与其所匹配的关键词进行汇总，将抽象的主题模型转化为具体、直观的形式，汇总信息详见表 4-1：

表 4-1：主题及其关键词汇总表

主题	关键词
儿童启蒙类	喜欢，孩子，好书，质量，值得，好好，推荐，数学，收到，购买，希望，东西，好看，有用，思维，3d，很大，好玩，数学题，好好学习
购物体验类	不错，本书，内容，包装，满意，感觉，好评，实用，阅读，正版，印刷，精美，有意思，工作，慢慢，知识，当当，图书，专业性，纸张
学习深度类	学习，python，很快，有趣，快递，入门，书籍，深度，经典，一本，速度，谜题，数学，物流，真的，通俗易懂，适合，初评，基础，儿子

见表 4-1，出版社可以根据汇总表中的关键词，策划有针对性的选题，如结合“3D”“好玩”等关键词，开发具有立体效果或互动性强的儿童读物。关注购物体验类的关键词，合理地优化自己的图书质量，提高读者的体验感。依据学习深度类关键词可以打造系列化、专业化的学习图书。

作者可以在创作中融入“孩子”“喜欢”等元素，以儿童为中心，采用生动有趣的语言和故事形式，让内容更具吸引力。了解购物体验类关键词，在写作中注重图书内容的实用性和可读性，以提高读者的满意度。或者结合学习深度类关键词，深入研究专业领域知识，确保内容准确、权威。

4.3.3 热销书用户评论情感分析

主要围绕图书评论数据展开深度挖掘与分析，通过系统化的数据处理流程，精准识别用户评论中的好评与差评，并提炼出具有代表性的正面与负面关键词。在图书质量方面，出版社可以针对负面关键词所反映的问题，如内容深度不足、图书质量差等，进行内容优化与质量把控，若发现“运费”、“退货”等负面高频词，则可与物流协商优化退换货以及运输方案，提升物流服务的效率与质量。通过对评论数据的分析与关键词提取，出版社得以实现以用户反馈为导向的精准优化，从而提升用户满意度与市场竞争力。

1. 特征与目标选择

在这部分对特征和目标进行选择，用户的评论内容和评分很大程度上代表着用户对图书的满意程度，用评论内容和评分进行情感分析是比较合适的选择。如果仅仅使用评论内容进行分析，则可能准确率比较低，如果仅仅使用评分进行分析，则只知道用户打分多少，不知道具体的原因，需要两者结合分析，结果才更加科学可靠。

X: 评论内容（即文本特征），y: 情感标签，如果 y≥5 则划分为 1，即为正面情感，否则为 0，即为负面情感。随后计算出正面情感评论和负面情感评论的数量，为后续分析做准备。

具体代码详见代码清单 4-23：

代码清单 4-23

```
# 选择特征和目标
X = data['评论内容']
y = data['评分']
# 将评分大于等于 5 分的标记为正面情感(1)，小于 5 分的标记为负面情感(0)
y = (y >= 5).astype(int)
# 计算正面和负面评论的数量
sentiment_counts = y.value_counts()
print(f"正面评论数量: {sentiment_counts.get(1, 0)}")
print(f"负面评论数量: {sentiment_counts.get(0, 0)}")
```

进行文本向量化：首先使用 TF-IDF 将文本转换为数值特征，并且限制特征数量为 5000 个最重要的词，能够防止特征维度爆炸。

具体代码详见代码清单 4-24：

代码清单 4-24

```
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
X_tfidf = tfidf_vectorizer.fit_transform(X)
```

2.模型训练与评估

划分训练集和测试集：这里按照 8:2 的比例对数据进行随机的分割，利用`random_state`来确保可重复性。

具体代码详见代码清单 4-25：

代码清单 4-25

```
X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.2, random_state=42)
```

训练 SVM 模型：使用线性核的 SVM 分类器，适用于文本分类，并且训练模型。

具体代码详见代码清单 4-26：

代码清单 4-26

```
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train, y_train)
```

预测和评估：在训练集和测试集上进行预测，然后计算以下四个评估的指标：准确率(ACC)是指正确预测的比例，F1 分数是指精确率和召回率的调和平均，精确率(P)是指预测为正的样本中实际为正的比例，召回率(R):是指实际为正的样本中被正确预测的比例。

具体代码详见代码清单 4-27：

代码清单 4-27

```
y_train_pred = svm_model.predict(X_train)
y_test_pred = svm_model.predict(X_test)
# 计算训练集和测试集的评估指标
def evaluate_metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    return acc, f1, precision, recall
```

模型评估具体内容详见图 4-24：

```
正面评论数量: 3203
负面评论数量: 86
训练集评估指标:
准确率 (ACC): 0.9806
F1分数 (F1): 0.9901
精确率 (P): 0.9805
召回率 (R): 1.0000

测试集评估指标:
准确率 (ACC): 0.9787
F1分数 (F1): 0.9892
精确率 (P): 0.9802
召回率 (R): 0.9984
```

图 4-24：模型训练评估结果展示图

3.关键词提取

本部分将关键词数据分为了正面关键词和负面关键词，首先先获取了正面和负面的关键词的特征，正面关键词获取最积极情感的 50 个词，负面关键词获取最消极情感的 50 个词。按照从最重要到最不重要进行排序并打印。

具体代码详见代码清单 4-28：

代码清单 4-28

```
# 获取正面和负面的关键词特征
pos_indices = np.argsort(coef)[-50:]  # 最积极情感的 50 个词
neg_indices = np.argsort(coef)[:50]   # 最消极情感的 50 个词
# 正面关键词
pos_keywords = [(feature_names[i], coef[i]) for i in pos_indices]
pos_keywords.reverse()  # 从最重要到最不重要
# 负面关键词
neg_keywords = [(feature_names[i], coef[i]) for i in neg_indices]
# 打印关键词
print("\n 正面情感关键词（Top 20）:")
for keyword, weight in pos_keywords[:20]:
    print(f"{keyword}: {weight:.4f}")
print("\n 负面情感关键词（Top 20）:")
for keyword, weight in neg_keywords[:20]:
    print(f"{keyword}: {weight:.4f}")
```

根据获取到的数据信息绘制出统计图，正面情感关键词具体内容详见图 4-25：

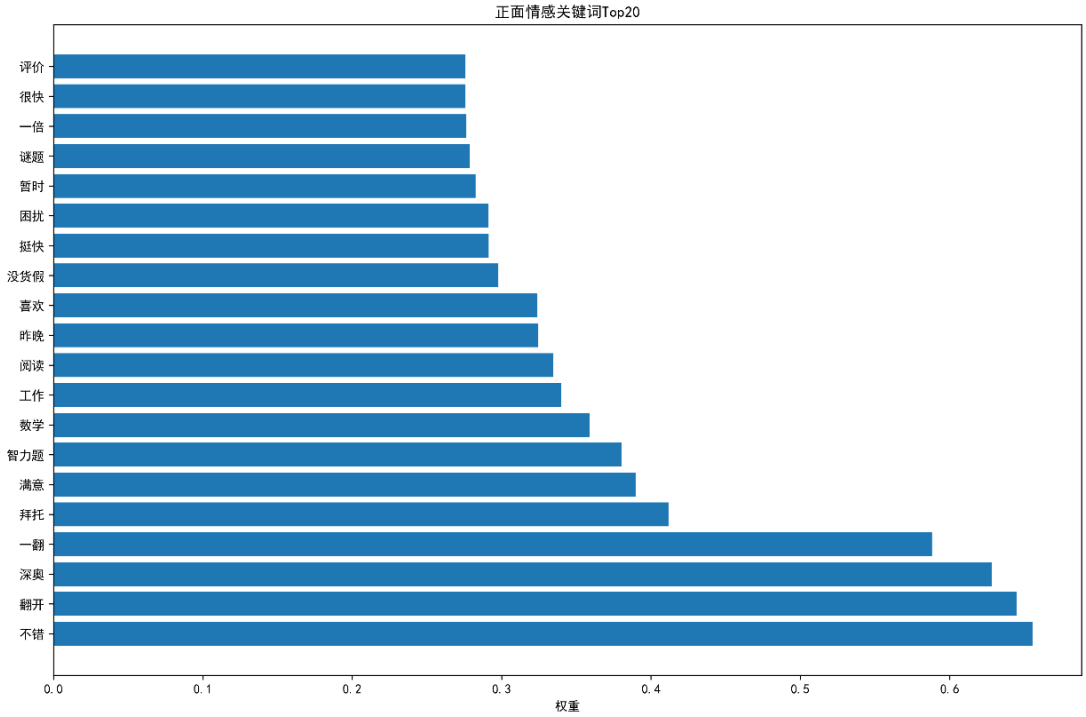


图 4-25：正面情感关键词 Top20 展示图

见图 4-25，纵轴为各个关键词，而横轴代表权重，可以看出用户评论最多的关键词为“不错”，所占权重最大，说明在评论中，“不错”是一个非常重要的关键词。其他正面情感高频关键词依次为翻开、深奥、一翻、拜托、满意、智力题、数学、工作、阅读等等

负面情感关键词具体内容详见图 4-26：

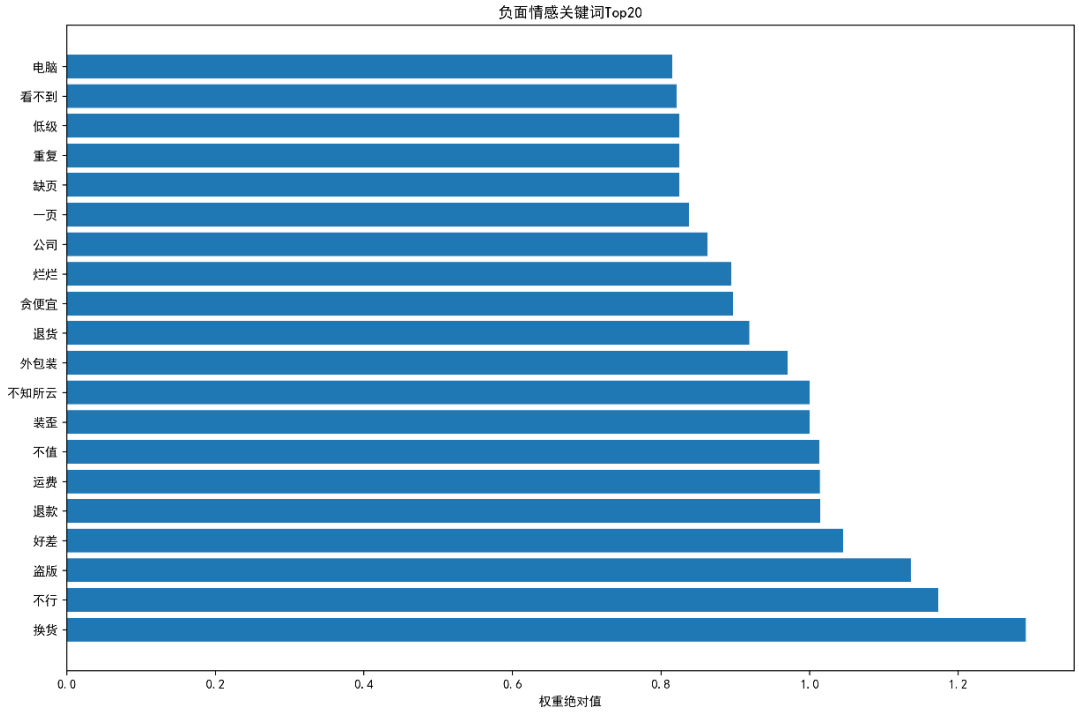


图 4-26：负面情感关键词 Top20 展示图

见图 4-26，“换货”、“不行”、“盗版”等等关键词所占权重非常大，这些关键词在评论中有很大的重要性。其他负面情感高频关键词依次是好差、退款、运费、不值、装歪、不知所云等等。

针对“不行”、“好差”、“不知所云”这类关键词应当注意多关注图书的质量问题，加强工厂的品控和质检，定期抽检商品来保证产品的合格率。针对“装歪”这种关键词应当优化包装，或者增加验货服务避免纠纷。针对“退款”、“退货”这种关键词应该提供更加良好的售后保障，制定明确的退换货政策，并且对于质量问题应该主动赔偿。

基于评论数据中挖掘出的负面情感词，平台需构建良好的改进机制，以实现精准优化与服务升级。通过对比优化前后的负面情感关键词词变化，验证改进措施的有效性，这样能显著降低产品差评率，进而提升用户口碑与市场竞争力。

第 5 章 总结

本研究使用网络爬虫技术，爬取了国内知名图书电商平台当当网的 Python 类图书销售相关数据以及用户评论数据，包括书名、作者、出版社、定价、评论数量等关键字段，考虑到原始数据存在格式不统一、缺失值、异常值以及重复记录等问题，采用了数据清洗、缺失值填充、异常值处理、文本规范化等多种技术手段，对采集到的数据进行了系统性的预处理工作，确保数据的完整性、准确性与可用性。

在出版社数据分析与可视化部分中，使用了 Matplotlib 以及 Seaborn 数据可视化库进行可视化，对出版社的读者满意度、定价、销量、出版量进行了分析，并采用相关性算法计算了星级、价格、用户评论这三个特征值之间的相关性。能够更深入地了解到各个变量之间更深层的关系，能够在一定程度上为平台、电商以及用户提供决策依据。

在图书销量数据分析中，首先对作者进行分析研究，其中包括高产作者分析、高热度作者分析以及高质量作者分析。对应用领域进行了分析，包括应用领域分布分析、各应用领域热度分析以及各应用领域满意度分析，并对图书销量采用了相关性算法来探究图书销量与各特征值之间的关系。随后对多种模型进行训练与评估并选择出最佳模型为梯度提升，根据相关性最大的几个特征值使用了梯度提升对销量进行预测，能够帮助出版社优化自己的定价策略与营销方法。

在热销图书用户评论情感分析部分，首先运用了 K-means 聚类算法对商品数据进行分类，挖掘出热销商品，并利用热销商品的用户评论数据进行情感分析。利用 LDA 进行了主题建模，通过分析困惑度、一致性、对数似然度这三个指标确定了最佳主题数，并提取了儿童启蒙类、购物体验类、学习深度类这三个主题所对应的关键词。并在此基础上，利用 TF-IDF + SVM 机器学习模型先对评论的好评与差评进行挖掘分析，提取出正面和负面的关键词，为商品优化和服务改进提供有力的数据支持。

参考文献

[1] 唐磊. 基于 Selenium 和 Ddddocr 技术自动化网络爬虫设计与实现[J]. 电脑编程技巧与维护, 2025, (02): 161-163.

[2] 李许, 李珺, 麦宝华, 等. 基于网络爬虫和大数据技术的技术性贸易措施应对研究[J]. 中国口岸科学技术, 2025, 7(02): 10-14.

[3] 汪广龙, 黄浩天. 网络空间中群团工作的嵌入机制——基于 B 站合创视频的爬虫与文本分析[J]. 学术研究, 2025, (02): 66-72.

[4] 母爱芹. 基于 Python 的爬虫技术抓取岗位任职需求的可视化研究[J]. 信息记录材料, 2025, 26(02): 132-134.

[5] 余华东. 基于爬虫的网络文本挖掘高职院校电子商务专业课程设置分析与研究[J]. 铜陵职业技术学院学报, 2024, 23(04): 5-10.

[6] 熊建芳, 冯雯, 高继, 等. 基于线性回归气象数据预测及可视化系统设计与实现[J]. 现代信息科技, 2024, 8(23): 133-137+144.

[7] 崔晓青. 基于 Python 爬虫的招聘信息采集与分析——以天津电子信息职业技术学院为例[J]. 电脑知识与技术, 2024, 20(34): 57-59+69.

[8] 曹骋. 基于 Python 的材料力学教学数据可视化方法探讨[J]. 信息与电脑, 2025, 37(08): 173-175.

[9] 卢亮, 韩彩虹, 郭秀凤. 基于 Python 的数学分析课程可视化和计算教学探索[J]. 电脑知识与技术, 2024, 20(33): 6-10.

[10] 赵志凡, 邓一哲, 张思源 ,等. 基于 Python 的城市天气数据可视化分析[J]. 软件, 2024, 45(04): 37-39.

[11] 杨冰倩. 基于 Python 爬虫的影评情感分析与可视化系统设计[J]. 无线互联科技, 2023, 20(20): 43-45+49.

[12] 黄正鹏, 马欣, 陈雪, 等. 基于线性回归算法对多彩贵州旅游数据的分析及应用[J]. 软件工程, 2024, 27(03): 63-66.

[13] 吕翔. 基于 Python 的山东省小麦赤霉病预测模型研究[D]. 山东农业大学, 2023.

[14] 陈旭. 基于文本聚类的用户评论热点挖掘[D]. 安徽财经大学, 2020.

[15] 马骁. 基于数据采集的电商评论分析系统的研究[D]. 青岛科技大学, 2018.

[16] 索喜泽. 基于网络爬虫技术的交通热点事故信息获取及其舆情分析[D]. 昆明理工大学, 2020.

[17] 罗攀. 从零开始学 Python 数据分析[M]. 机械工业出版社: 202302.635.

[18] 孙玉林. Python 机器学习[M]. 化学工业出版社: 202308.324.

[19] 王宇韬, 钱妍竹. Python 大数据分析与机器学习商业案例实战[M]. 机械工业出版社: 202301.787.

[20] 屈希峰. Python 数据可视化[M]. 机械工业出版社: 202201.521.

致谢

非常感谢在 Python 语言类图书销售和评论数据分析研究中帮助过我的老师和同学，更要感激指导老师对我尽心尽力的辅导，帮我指出数据中存在的问题和分析过程中的不足，认真负责地督促我完成论文。从最初的选题到获取数据、数据分析、编写论文等方方面面都给予我最大的鼓励和耐心的指导，对我有着非常大的帮助和鼓舞。

同时也非常感谢所有参加评审和答辩的老师们，给我这样一个考核和检验自己大学学习成果的机会，让我更加清晰地认识到自己的不足，明确自己未来的发展，为我的大学生活画上一个完美的句号，这些经历都将会成为我人生中宝贵的财富。